

Nama : Kolonel Gallih Bimantara

NPM : 23312199

Jobdesk : api, user

DOKUMENTASI

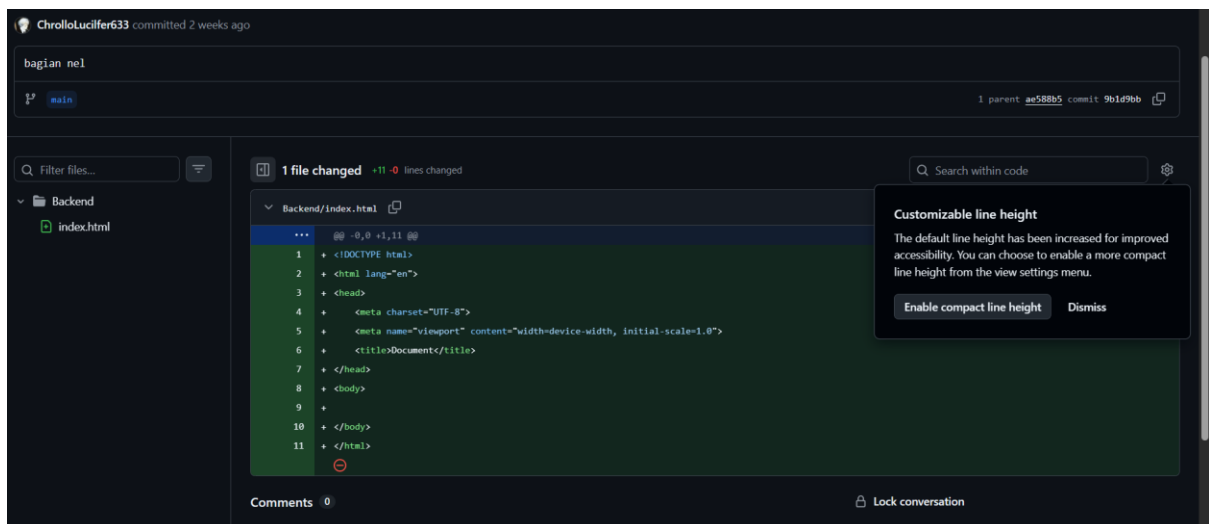
1. Commits on April 4, 2026



Pada tahap ini saya melakukan inisialisasi repositori untuk pertama kalinya saya mengunggah file dasar yang akan digunakan dalam proyek.

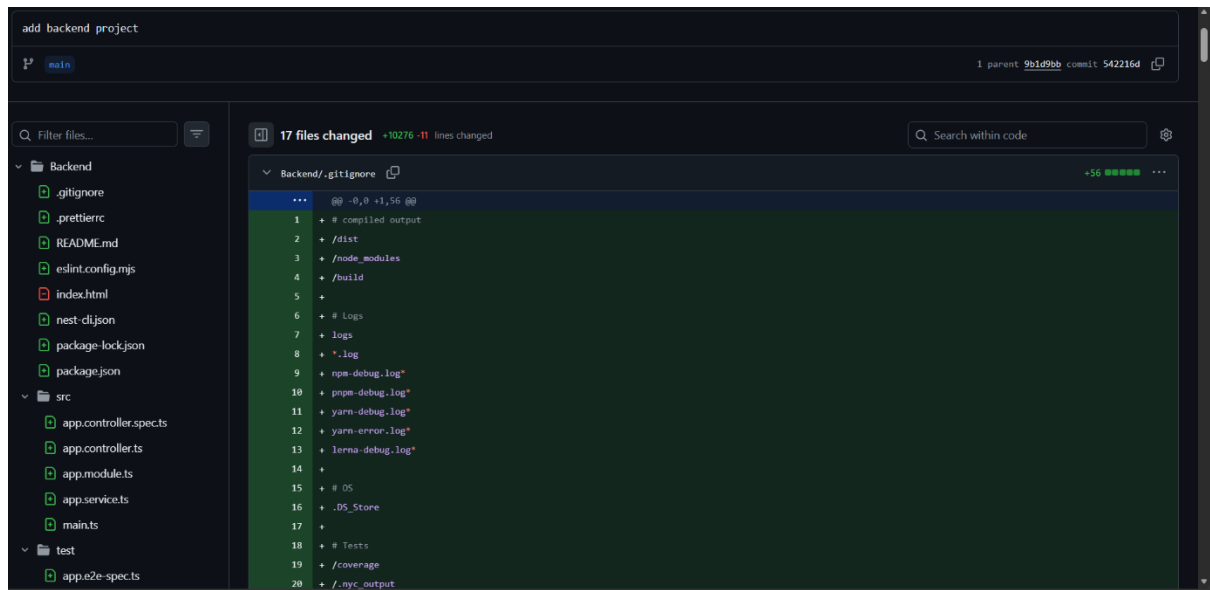
2. Commits on April 5, 2026

a. Bagian kolonel



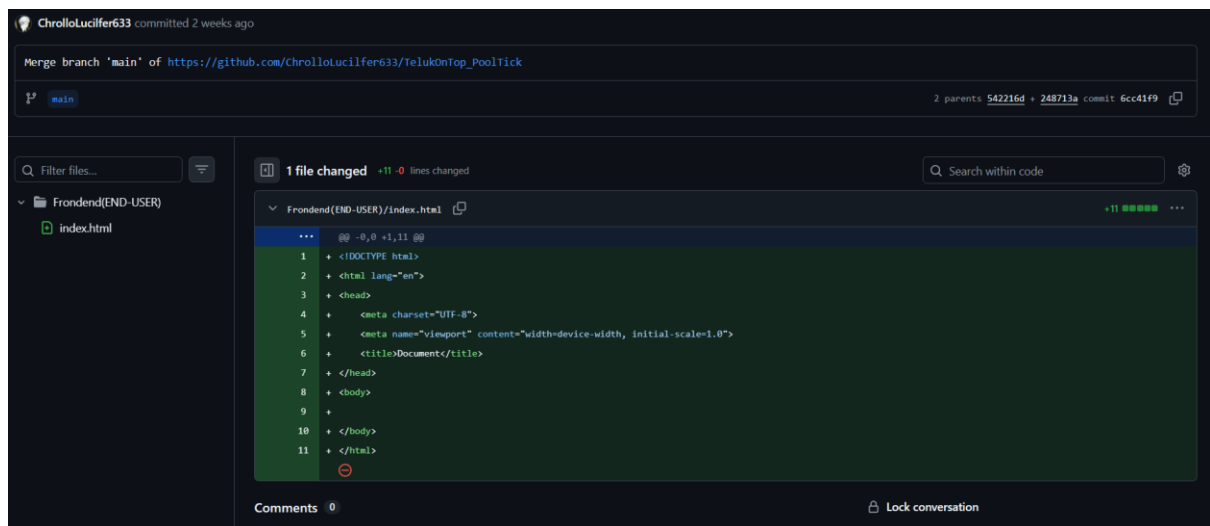
Pada tahap ini, saya membuat file index html dulu di dalam folder backend

b. Add backend project



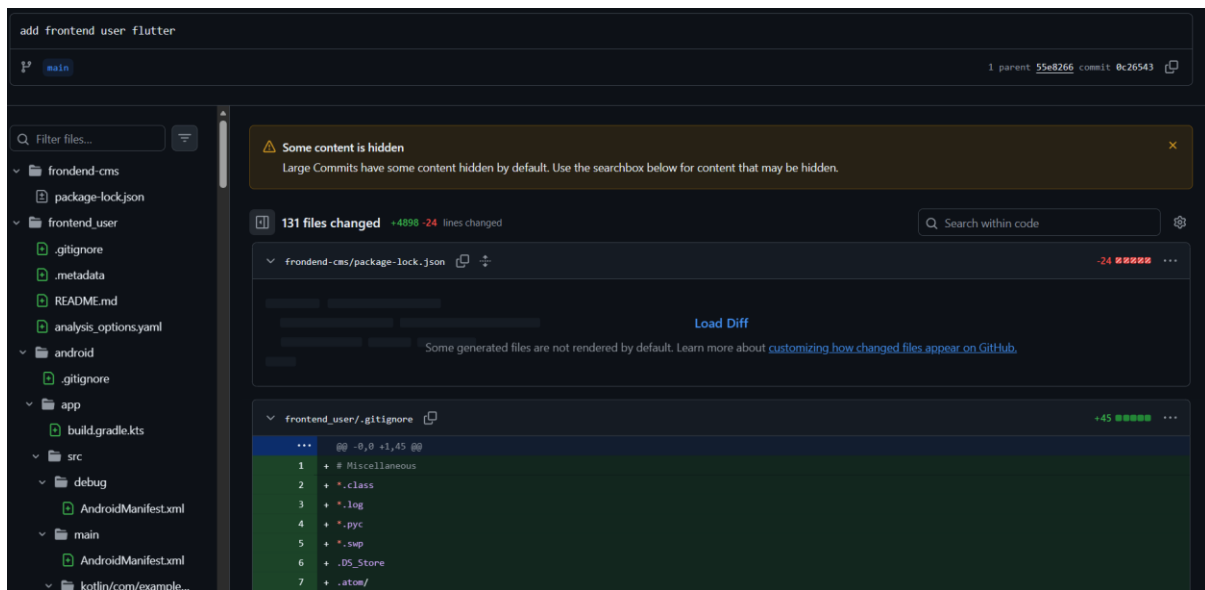
mengonfigurasi file .gitignore untuk memastikan file sampah atau folder berat seperti node_modules tidak ikut terunggah ke repositori, selain itu, saya menyiapkan struktur folder standar Node.js yang mencakup file konfigurasi seperti package.json, eslint, dan prettier guna menjaga kualitas dan konsistensi kode program

c. Merge branch 'main' of https://github.com/ChrolloLucifer633/TelukOnTop_PoolTick



Disini saya melakukan merge branch untuk menggabungkan perubahan terbaru ke cabang utama. Dalam proses ini, saya menambahkan struktur dasar Frontend(END-USER)

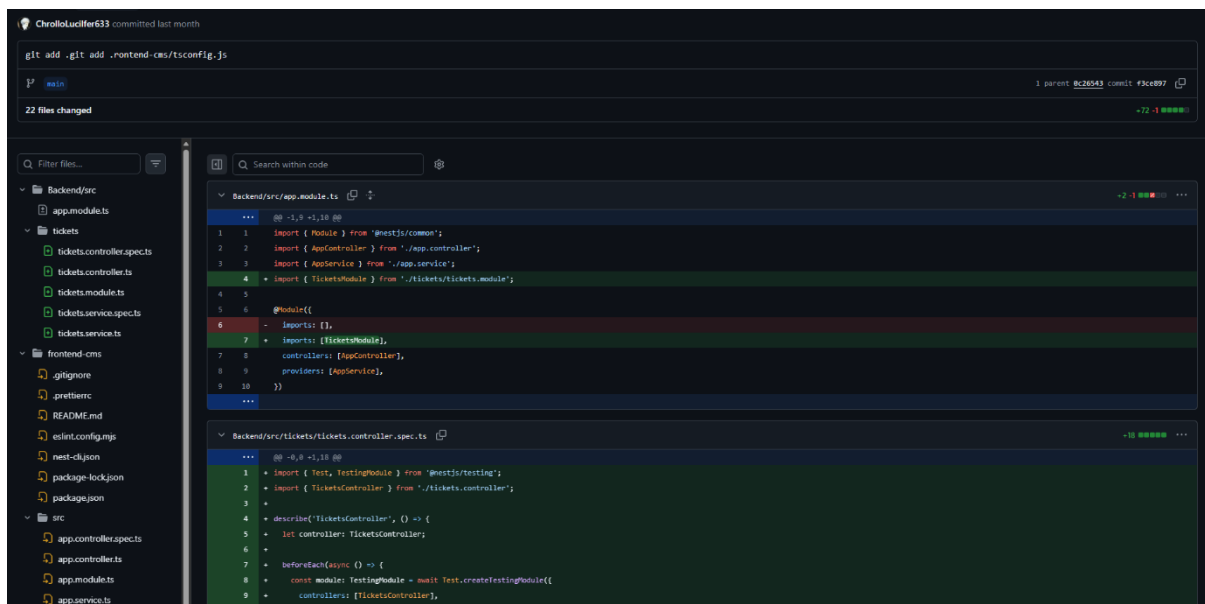
d. add frontend user flutter



Pada tahap ini, saya menambahkan proyek Frontend User menggunakan Flutter framework saya menginisialisasi struktur aplikasi mobile yang mencakup konfigurasi untuk platform Android, file analysis_options.yaml untuk standarisasi kode, serta file .gitignore khusus Flutter guna memastikan file build system tidak mengotori repostori. Langkah ini bertujuan untuk memulai pengembangan antarmuka pengguna berbasis aplikasi mobile

3. Commits on April 6, 2026

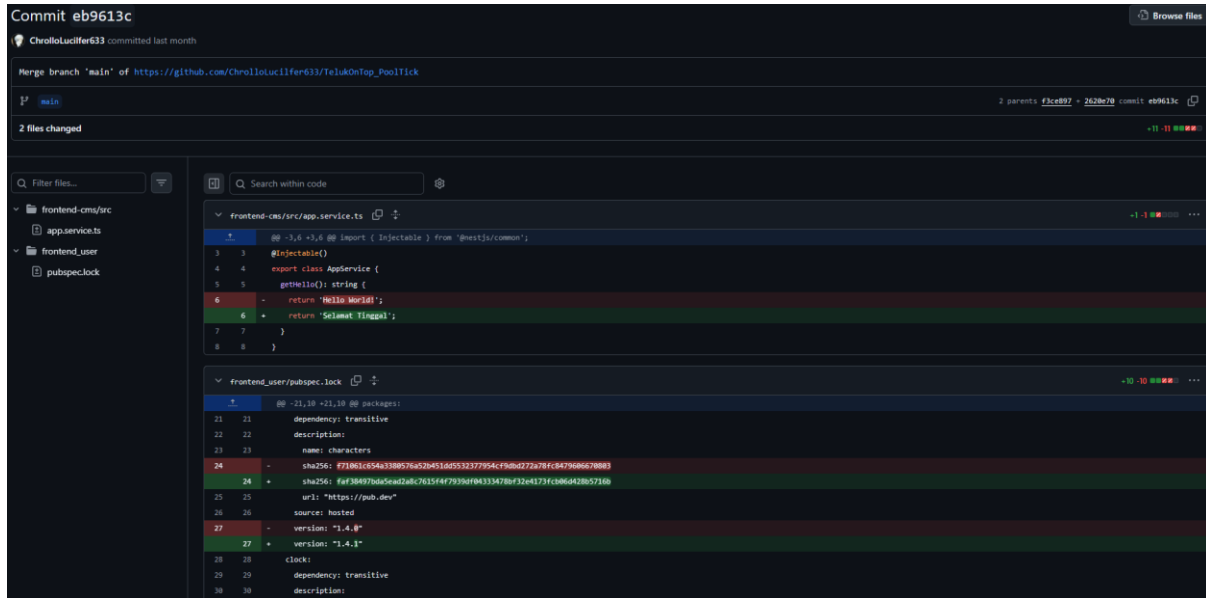
a. [git add .git add .rontend-cms/tsconfig.js](#)



Pada bagian ini, saya mengembangkan fitur Tickets di sisi Backend menggunakan NestJS. Saya membuat modul baru bernama TicketsModule dan mendaftarkannya ke dalam AppModule agar fitur tersebut dapat berjalan di dalam aplikasi. Selain itu, saya juga menyusun

TicketsController beserta file pengujiannya (spec.ts) untuk menangani logika API terkait sistem pertiketan secara terstruktur

b. Merge branch 'main'
of https://github.com/ChrolloLucifer633/TelukOnTop_PoolTick



```
Commit eb9613c
ChrolloLucifer633 committed last month

Merge branch 'main' of https://github.com/ChrolloLucifer633/TelukOnTop_PoolTick

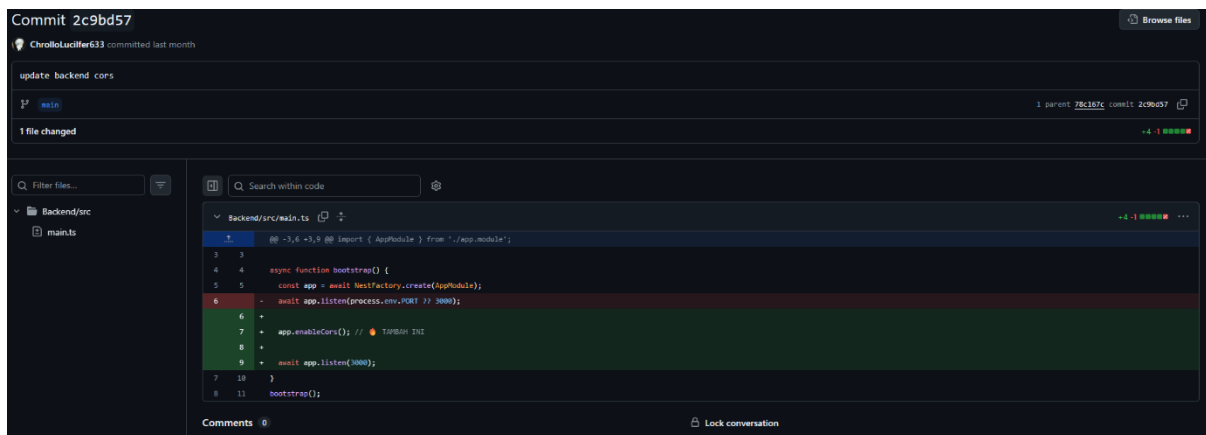
2 files changed
+11 -11

frontend-cms/src/app.service.ts
3 3  @Injectable()
4 4  export class AppService {
5 5  getHello(): string {
6 -   return 'Hello World!';
6 +   return 'Selamat Tinggal';
7 7  }
8 8  }

frontend_user/pubspec.lock
21 21  dependency: transitive
22 22  description:
23 23  name: characters
24 - sha256: #21861c54a3389c76a520451d4553277954cf9b4d772a78fc947968670883
24 + sha256: f4f384973da5ead2a8763564f7939af04335478bf3244373fc086d428057169
25 25  url: "https://pub.dev"
26 26  source: hosted
27 - version: "1.4.0"
27 + version: "1.4.1"
28 28  clock:
29 29  dependency: transitive
30 30  description:
```

pada commit ini, saya melakukan update pesan service dan pembaruan dependensi. Di bagian backend, saya mengubah *return value* pada fungsi `getHello()` di file `app.service.ts` dari 'Hello World!' menjadi 'Selamat Tinggal'. Selain itu, saya melakukan pembaruan versi package pada aplikasi Flutter melalui file `pubspec.lock` (dari versi 1.4.0 ke 1.4.1) untuk memastikan dependensi proyek tetap mutakhir

c. update backend cors



```
Commit 2c9bd57
ChrolloLucifer633 committed last month

update backend cors

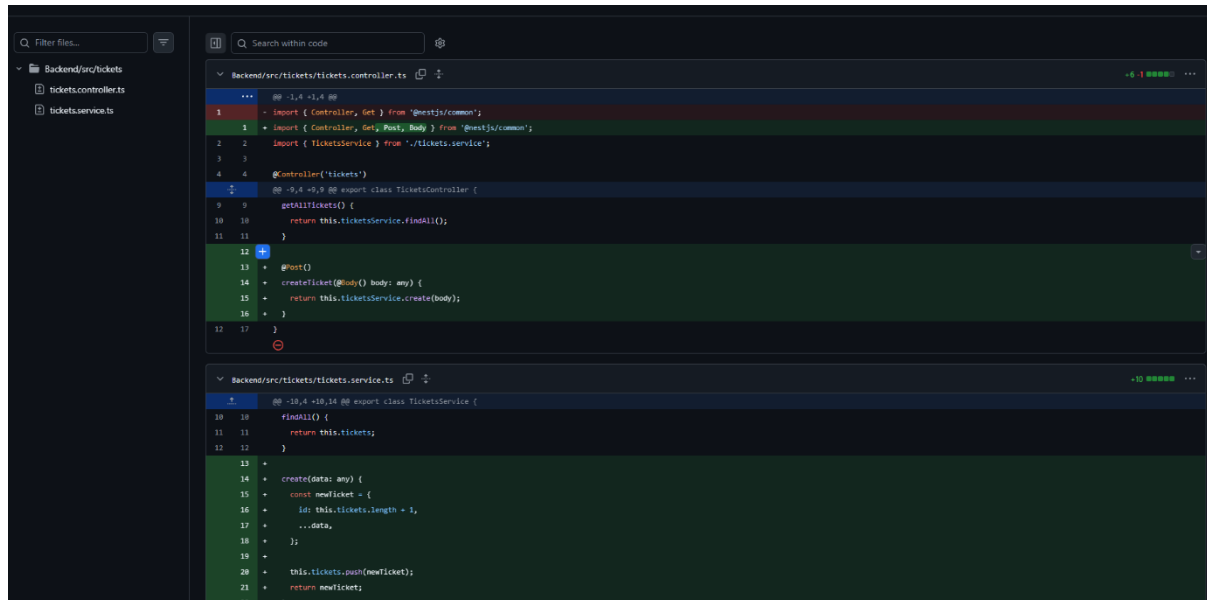
1 file changed
+4 -1

Backend/src/main.ts
1 1  @ -3,6 +3,8 @@ import { AppModule } from './app.module';
2 2
3 3
4 4  async function bootstrap() {
5 5  const app = await NestFactory.create(AppModule);
6 - await app.listen(process.env.PORT ?? 3000);
6 + app.enableCors(); // ENABLE CORS
7 +
8 + await app.listen(3000);
7 10  }
8 11  bootstrap();
```

Di sini saya melakukan konfigurasi CORS (Cross-Origin Resource Sharing) pada file `main.ts` di sisi Backend. Saya menambahkan perintah `app.enableCors()` agar server NestJS dapat menerima permintaan (request) dari domain luar, seperti aplikasi Frontend atau Mobile yang

sudah dibuat sebelumnya. Selain itu, saya juga menetapkan port aplikasi secara statis ke port 3000 untuk memastikan koneksi antar layanan berjalan stabil

d. Tambah fitur POST tiket

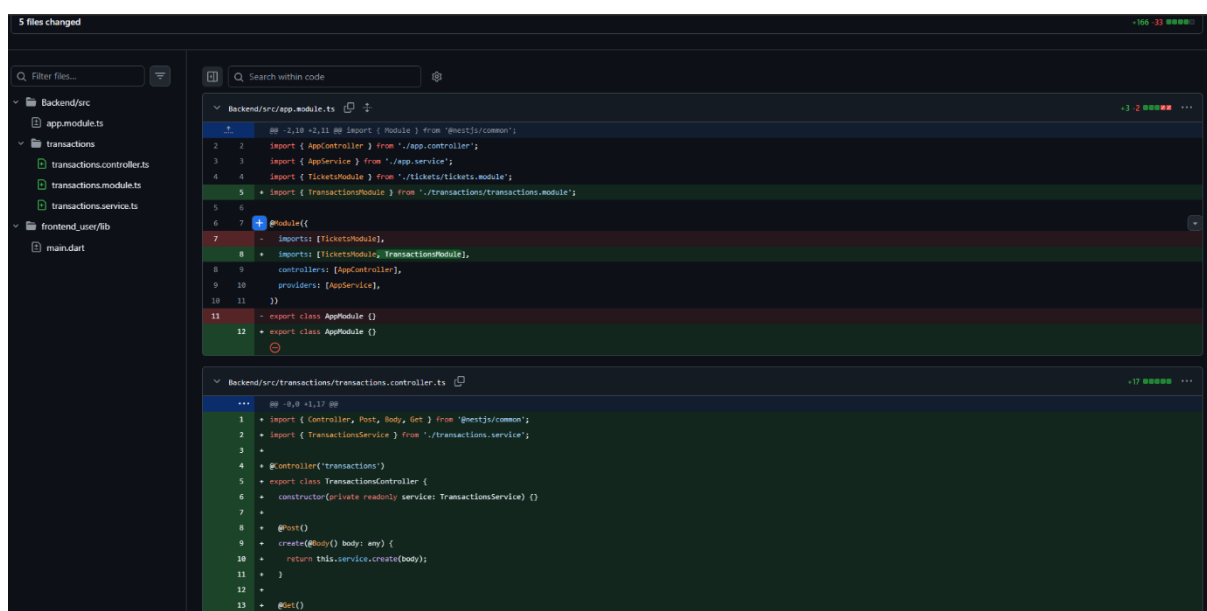


```
Backend/src/tickets/tickets.controller.ts
1 // @ 1.4 - 1.4 @@
2 import { Controller, Get } from '@nestjs/common';
3 import { Controller, Get, Post, Body } from '@nestjs/common';
4 import { TicketsService } from './tickets.service';
5
6 @Controller('tickets')
7 // @ 1.4 - 1.4 @@ export class TicketsController {
8
9   getAllTickets() {
10     return this.ticketsService.findAll();
11   }
12
13   @Post()
14   createTicket(@Body() body: any) {
15     return this.ticketsService.create(body);
16   }
17 }

Backend/src/tickets/tickets.service.ts
1 // @ 1.4 - 1.4 @@ export class TicketsService {
2
3   findAll() {
4     return this.tickets;
5   }
6
7   create(data: any) {
8     const newTicket = {
9       id: this.tickets.length + 1,
10       ...data,
11     };
12     this.tickets.push(newTicket);
13     return newTicket;
14   }
15 }
```

Pada tahap ini, saya menambahkan fitur pembuatan tiket baru pada sisi Backend. Di dalam tickets.controller.ts, saya menambahkan dekorator `@Post()` dan method `createTicket` untuk menangani permintaan data dari user. Secara bersamaan, saya juga memperbarui tickets.service.ts dengan logika untuk memproses data tersebut, di mana sistem akan otomatis membuat ID unik berdasarkan panjang data yang ada dan menyimpannya ke dalam array tickets

e. Tambah fitur transaksi + flutter beli tiket



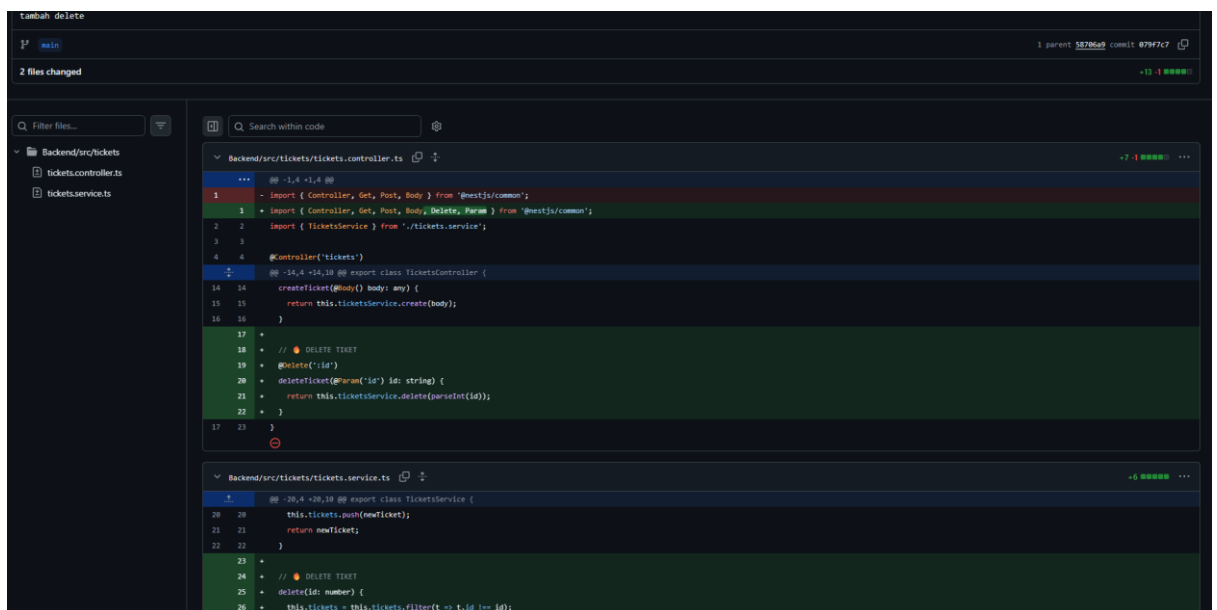
```
Backend/src/app.module.ts
1 // @ 1.18 - 1.18 @@ import { Module } from '@nestjs/common';
2 import { AppController } from './app.controller';
3 import { AppService } from './app.service';
4 import { TicketsModule } from './tickets/tickets.module';
5 import { TransactionsModule } from './transactions/transactions.module';
6
7 @Module({
8   imports: [TicketsModule, TransactionsModule],
9   controllers: [AppController],
10   providers: [AppService],
11 })
12 export class AppModule {}
13 export class AppModule {}

Backend/src/transactions/transactions.controller.ts
1 // @ 1.17 - 1.17 @@
2 import { Controller, Post, Get } from '@nestjs/common';
3 import { TransactionsService } from './transactions.service';
4
5 @Controller('transactions')
6 export class TransactionsController {
7   constructor(private readonly service: TransactionsService) {}
8
9   @Post()
10   create(@Body() body: any) {
11     return this.service.create(body);
12   }
13
14   @Get()
15   findAll() {
16     return this.service.findAll();
17   }
18 }
```

Pada tahap ini, saya mengimplementasikan fitur Transactions pada sisi Backend. Saya membuat folder baru bernama transactions yang berisi TransactionsModule, TransactionsController, dan TransactionsService. Saya juga mendaftarkan TransactionsModule ke dalam AppModule agar fitur transaksi ini aktif di sistem. Controller yang baru dibuat ini sudah mendukung method `@Post()` untuk membuat transaksi baru dan `@Get()` untuk mengambil data transaksi yang ada

4. Commits on April 8, 2026

a. Tambah delete

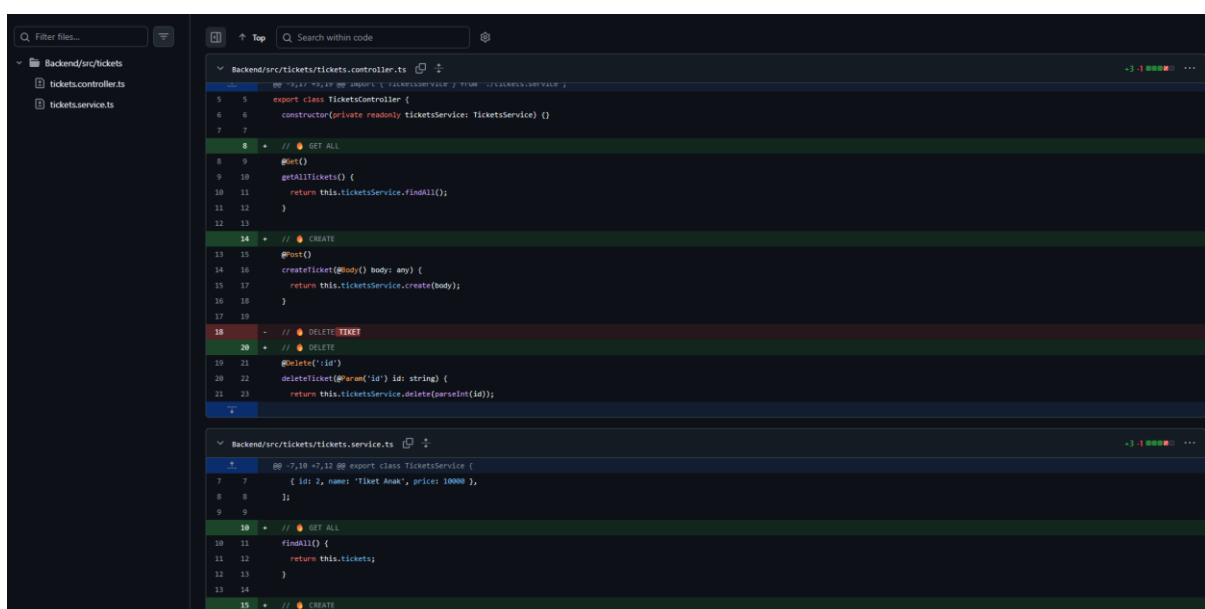


```
Backend/src/tickets/tickets.controller.ts
1  import { Controller, Get, Post, Body } from '@nestjs/common';
2  import { Controller, Get, Post, Delete, Param } from '@nestjs/common';
3  import { TicketsService } from './tickets.service';
4
5  @Controller('tickets')
6
7  @ -14,4 +14,10 @@ export class TicketsController {
14 14  createTicket(@Body() body: any) {
15 15  return this.ticketsService.create(body);
16 16  }
17
18  // DELETE TICKET
19  @Delete('/:id')
20  deleteTicket(@Param('id') id: string) {
21  return this.ticketsService.delete(parseInt(id));
22  }
23  }

Backend/src/tickets/tickets.service.ts
1  @ -20,4 +20,10 @@ export class TicketsService {
20 20  this.tickets.push(newTicket);
21 21  return newTicket;
22 22  }
23
24  // DELETE TICKET
25  delete(id: number) {
26  this.tickets = this.tickets.filter(t => t.id !== id);
27  }
```

Pada tahap ini, saya menambahkan fitur hapus data (Delete) pada modul Tickets. Di dalam tickets.controller.ts, saya menambahkan endpoint dengan dekorator `@Delete('/:id')` agar sistem dapat menerima parameter ID tiket yang ingin dihapus. Secara sinkron, saya juga memperbarui tickets.service.ts dengan logika pemfilteran data, di mana sistem akan menghapus tiket dari daftar berdasarkan ID yang sesuai menggunakan fungsi `.filter()`

b. Hapus tiket

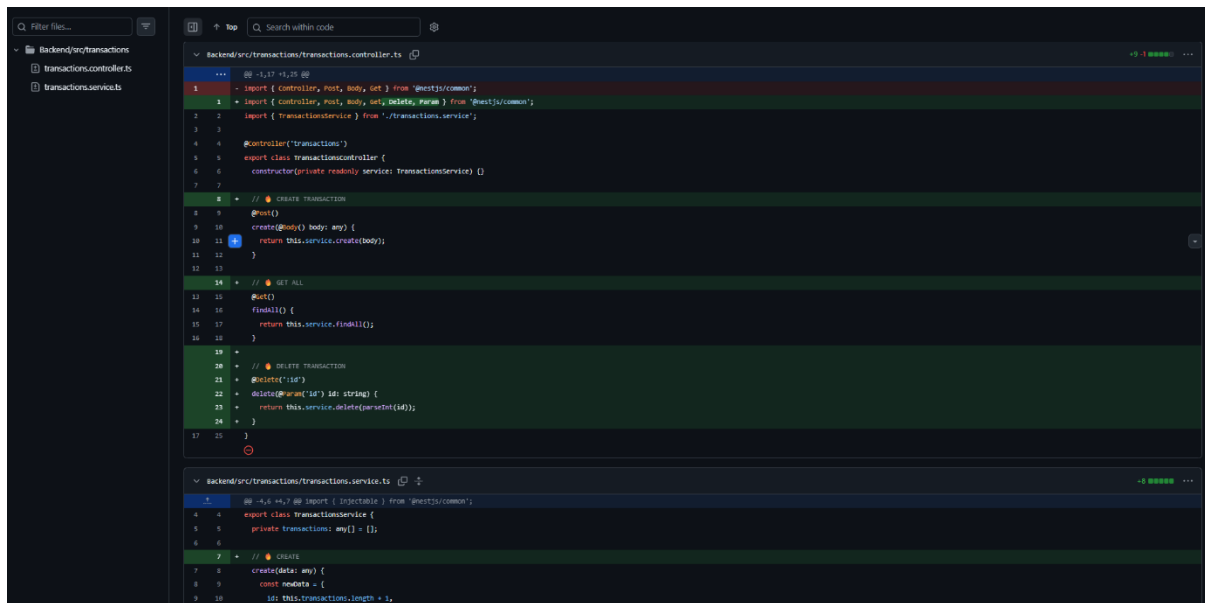


```
Backend/src/tickets/tickets.controller.ts
1  import { Controller, Get, Post, Body } from '@nestjs/common';
2  import { Controller, Get, Post, Delete, Param } from '@nestjs/common';
3  import { TicketsService } from './tickets.service';
4
5  @Controller('tickets')
6
7  @ -7,10 +7,12 @@ export class TicketsController {
7 7  constructor(private readonly ticketsService: TicketsService) {}
8 8
9 9  // GET ALL
10 10  @Get()
11 11  getAllTickets() {
12 12  return this.ticketsService.findAll();
13 13  }
14
15 15  // CREATE
16 16  @Post()
17 17  createTicket(@Body() body: any) {
18 18  return this.ticketsService.create(body);
19 19  }
20
21 21  // DELETE TICKET
22 22  @Delete('/:id')
23 23  deleteTicket(@Param('id') id: string) {
24 24  return this.ticketsService.delete(parseInt(id));
25 25  }

Backend/src/tickets/tickets.service.ts
1  @ -7,10 +7,12 @@ export class TicketsService {
7 7  ({ id: 2, name: 'Tiket Anak', price: 10000 },
8 8  );
9 9
10 10  // GET ALL
11 11  findAll() {
12 12  return this.tickets;
13 13  }
14
15 15  // CREATE
16 16  createTicket(body: any) {
17 17  this.tickets.push(body);
18 18  return body;
19 19  }
20
21 21  // DELETE
22 22  delete(id: number) {
23 23  this.tickets = this.tickets.filter(t => t.id !== id);
24 24  }
25 25  }
```

Pada bagian ini, saya melakukan perapian kode (refactoring) dan penambahan komentar dokumentasi pada modul Tickets. Saya menambahkan label penanda seperti GET ALL, CREATE, dan DELETE pada file tickets.controller.ts dan tickets.service.ts untuk mempermudah pembacaan alur logika program. Selain itu, saya memastikan setiap fungsi (mengambil semua data, menambah data, dan menghapus data) telah terorganisir dengan baik di dalam service agar kode lebih modular dan mudah dipelihara.

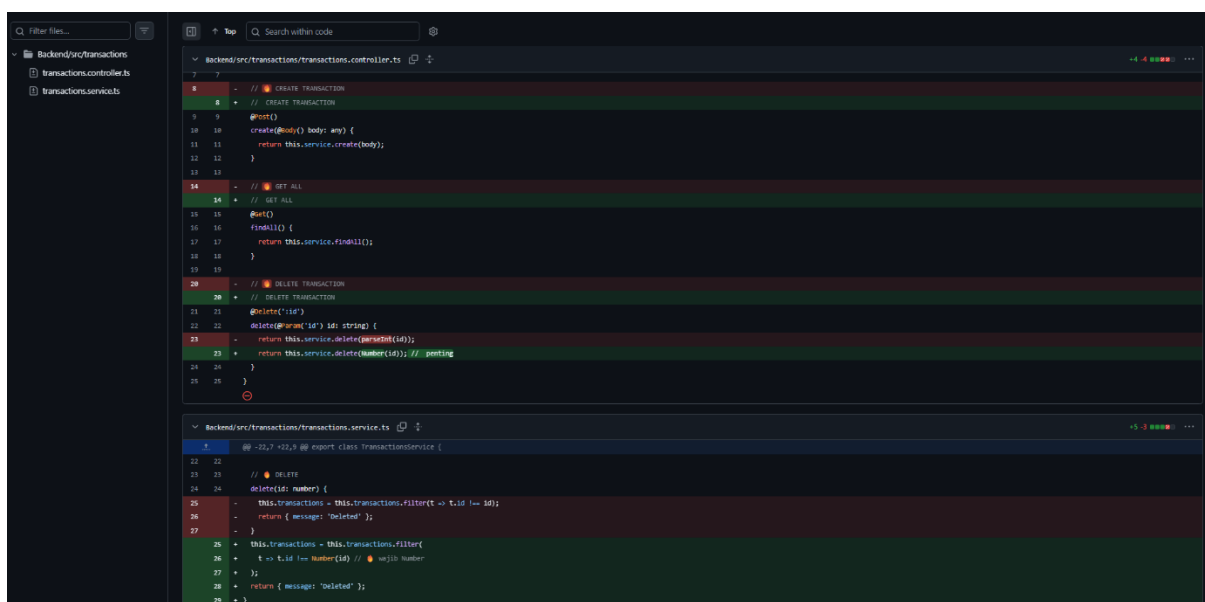
c. Menambahkan hapus transaksi



```
Backend/src/transactions/transactions.controller.ts
1 // @ts-ignore
2 import { Controller, Post, Body, Get } from '@nestjs/common';
3 import { TransactionService } from './transactions.service';
4
5 @Controller('transactions')
6 export class TransactionsController {
7   constructor(private readonly service: TransactionService) {}
8
9   // CREATE TRANSACTION
10  @Post()
11  create(@Body() body: any) {
12    return this.service.create(body);
13  }
14
15  // GET ALL
16  @Get()
17  findAll() {
18    return this.service.findAll();
19  }
20
21  // DELETE TRANSACTION
22  @Delete(':id')
23  delete(@Param('id') id: string) {
24    return this.service.delete(parseInt(id));
25  }
26 }
27
Backend/src/transactions/transactions.service.ts
1 // @ts-ignore
2 import { Injectable } from '@nestjs/common';
3
4 export class TransactionService {
5   private transactions: any[] = [];
6
7   // CREATE
8   create(data: any) {
9     const newData = {
10       id: this.transactions.length + 1,
11     };
12     this.transactions.push(newData);
13     return newData;
14   }
15
16   // DELETE
17   delete(id: number) {
18     this.transactions = this.transactions.filter(t => t.id !== id);
19     return { message: 'deleted' };
20   }
21
22   // GET ALL
23   findAll() {
24     return this.transactions;
25   }
26 }
```

Pada bagian ini, saya melengkapi fungsionalitas pada modul Transactions di sisi Backend. Saya menambahkan fitur Delete Transaction pada transactions.controller.ts menggunakan decorator `@Delete(':id')` dan memperbarui TransactionService untuk menangani penghapusan data berdasarkan ID. Selain itu, saya juga merapikan struktur kode dengan menambahkan komentar dokumentasi seperti CREATE TRANSACTION, GET ALL, dan DELETE TRANSACTION guna mempermudah identifikasi alur kerja API pada setiap fungsi transaksi

d. Membenarkan error di transactions



```
Backend/src/transactions/transactions.controller.ts
1 // @ts-ignore
2 import { Controller, Post, Body, Get } from '@nestjs/common';
3 import { TransactionService } from './transactions.service';
4
5 @Controller('transactions')
6 export class TransactionsController {
7   constructor(private readonly service: TransactionService) {}
8
9   // CREATE TRANSACTION
10  @Post()
11  create(@Body() body: any) {
12    return this.service.create(body);
13  }
14
15  // GET ALL
16  @Get()
17  findAll() {
18    return this.service.findAll();
19  }
20
21  // DELETE TRANSACTION
22  @Delete(':id')
23  delete(@Param('id') id: string) {
24    return this.service.delete(parseInt(id));
25  }
26 }
27
Backend/src/transactions/transactions.service.ts
1 // @ts-ignore
2 import { Injectable } from '@nestjs/common';
3
4 export class TransactionService {
5   private transactions: any[] = [];
6
7   // CREATE
8   create(data: any) {
9     const newData = {
10       id: this.transactions.length + 1,
11     };
12     this.transactions.push(newData);
13     return newData;
14   }
15
16   // DELETE
17   delete(id: number) {
18     this.transactions = this.transactions.filter(t => t.id !== id);
19     return { message: 'deleted' };
20   }
21
22   // GET ALL
23   findAll() {
24     return this.transactions;
25   }
26 }
```

Pada tahap final ini, saya melakukan optimasi tipe data pada fitur Delete Transaction. Saya mengubah fungsi `parseInt(id)` menjadi `Number(id)` di `transactions.controller.ts` dan `transactions.service.ts`. Hal ini dilakukan untuk memastikan bahwa parameter ID dikonversi menjadi tipe data `Number` secara eksplisit (`// wajib Number`), guna menghindari bug saat proses pemfilteran data transaksi yang akan dihapus

e. Update backend bagian transactions module

[illegible]

Pada tahap akhir ini, saya melakukan optimasi pada fitur Delete Transaction. Saya menambahkan konversi tipe data eksplisit menggunakan fungsi Number() pada parameter ID untuk memastikan akurasi saat proses penghapusan data. Selain itu, saya menambahkan logging console (console.log) pada file service untuk memudahkan proses *debugging* dalam memantau ID mana yang sedang diproses oleh sistem. Langkah ini memastikan bahwa integrasi antara Controller dan Service berjalan dengan stabil sebelum proyek masuk ke tahap pengujian akhir

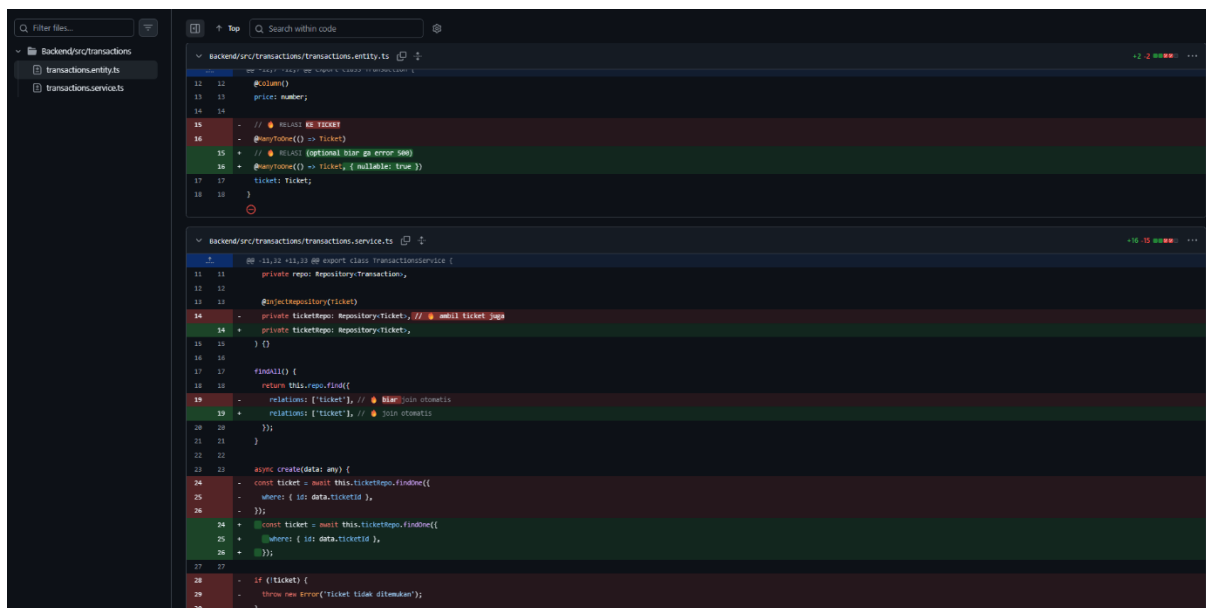
5. Commits on April 10, 2026

a. Membuat database di dbsql

[illegible]

Pada tahap ini, saya mengintegrasikan Database ke dalam sistem Backend. Saya menambahkan library TypeORM dan driver PostgreSQL (pg) ke dalam dependensi proyek melalui file package.json. Selanjutnya, saya mengonfigurasi koneksi database pada app.module.ts menggunakan TypeOrmModule.forRoot, yang mengatur koneksi ke server lokal PostgreSQL agar data tiket dan transaksi dapat disimpan secara permanen

b. Memperbaiki error di backend

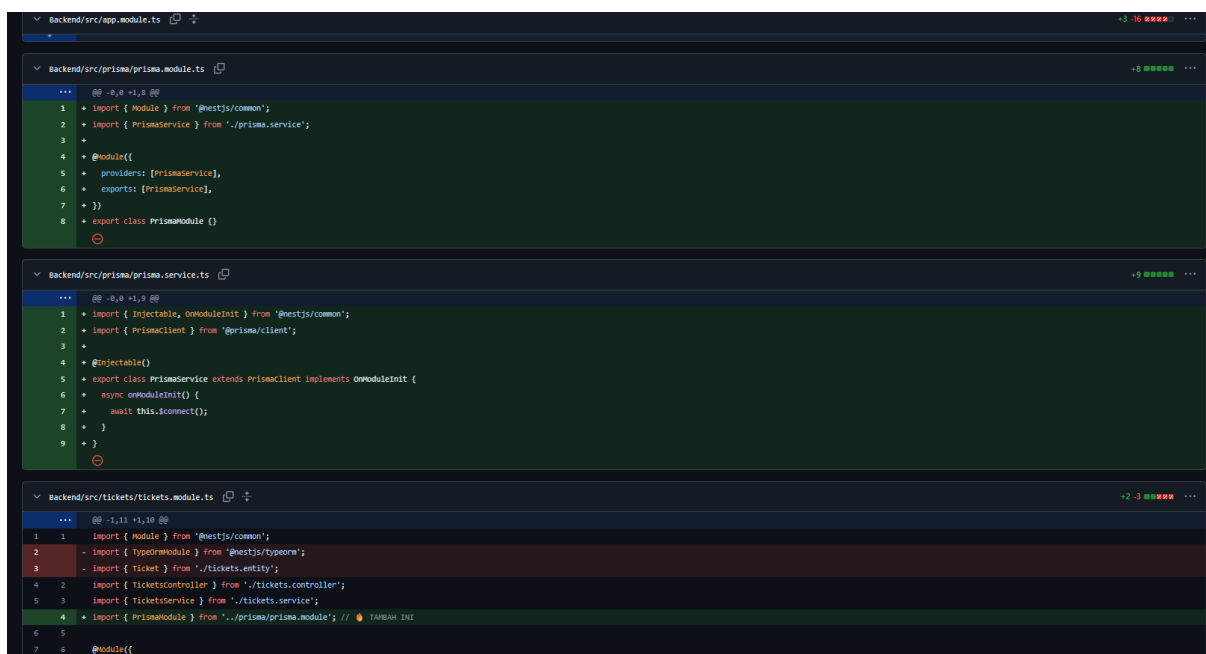


```
backend/src/transactions/transactions.entity.ts
12 12  @Column()
13 13  price: number;
14 14
15 15  - // RELASI KE TICKET
16 16  - @ManyToOne(() => Ticket)
17 17  + // RELASI (optional biar ga error 500)
18 18  + @ManyToOne(() => Ticket, { nullable: true })
19 19  ticket: Ticket;
20 20
21 21
22 22

backend/src/transactions/transactions.service.ts
7 7  @@ -11,12 +11,12 @@ export class TransactionService {
8 8  private repo: Repository<Transaction>;
9 9
10 10  @InjectRepository(Ticket)
11 11  - private ticketRepo: Repository<Ticket>; // @ todo ticket join
12 12  + private ticketRepo: Repository<Ticket>;
13 13
14 14  } {}
15 15
16 16  findAll() {
17 17  return this.repo.find({
18 18  relations: ['ticket'], // @ todo join otomatis
19 19  + relations: ['ticket'], // @ todo join otomatis
20 20  });
21 21  }
22 22
23 23  async create(data: any) {
24 24  - const ticket = await this.ticketRepo.findOne({
25 25  -   where: { id: data.ticketId },
26 26  - });
27 27  + const ticket = await this.ticketRepo.findOne({
28 28  +   where: { id: data.ticketId },
29 29  + });
30 30
31 31  - if (!ticket) {
32 32  -   throw new Error("Ticket tidak ditemukan");
33 33  - }
34 34
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100
```

Pada tahap ini, saya mengimplementasikan Relasi Database antara tabel Transaksi dan tabel Tiket. Di dalam transactions.entity.ts, saya menambahkan decorator @ManyToOne() dengan opsi nullable: true untuk menghubungkan transaksi ke data tiket guna menghindari error 500 jika data kosong. Selain itu, pada transactions.service.ts, saya menggunakan fitur Eager Loading/Join dengan perintah relations: ['ticket'] agar saat mengambil data transaksi, detail informasi tiket yang terkait otomatis ikut terbawa

c. Memperbaiki masalah data backend



```
backend/src/app.module.ts
1 1
2 2
3 3
4 4
5 5
6 6
7 7
8 8
9 9
10 10
11 11
12 12
13 13
14 14
15 15
16 16
17 17
18 18
19 19
20 20
21 21
22 22
23 23
24 24
25 25
26 26
27 27
28 28
29 29
30 30
31 31
32 32
33 33
34 34
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100

backend/src/prisma/prisma.module.ts
1 1
2 2
3 3
4 4
5 5
6 6
7 7
8 8
9 9
10 10
11 11
12 12
13 13
14 14
15 15
16 16
17 17
18 18
19 19
20 20
21 21
22 22
23 23
24 24
25 25
26 26
27 27
28 28
29 29
30 30
31 31
32 32
33 33
34 34
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100

backend/src/prisma/prisma.service.ts
1 1
2 2
3 3
4 4
5 5
6 6
7 7
8 8
9 9
10 10
11 11
12 12
13 13
14 14
15 15
16 16
17 17
18 18
19 19
20 20
21 21
22 22
23 23
24 24
25 25
26 26
27 27
28 28
29 29
30 30
31 31
32 32
33 33
34 34
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100

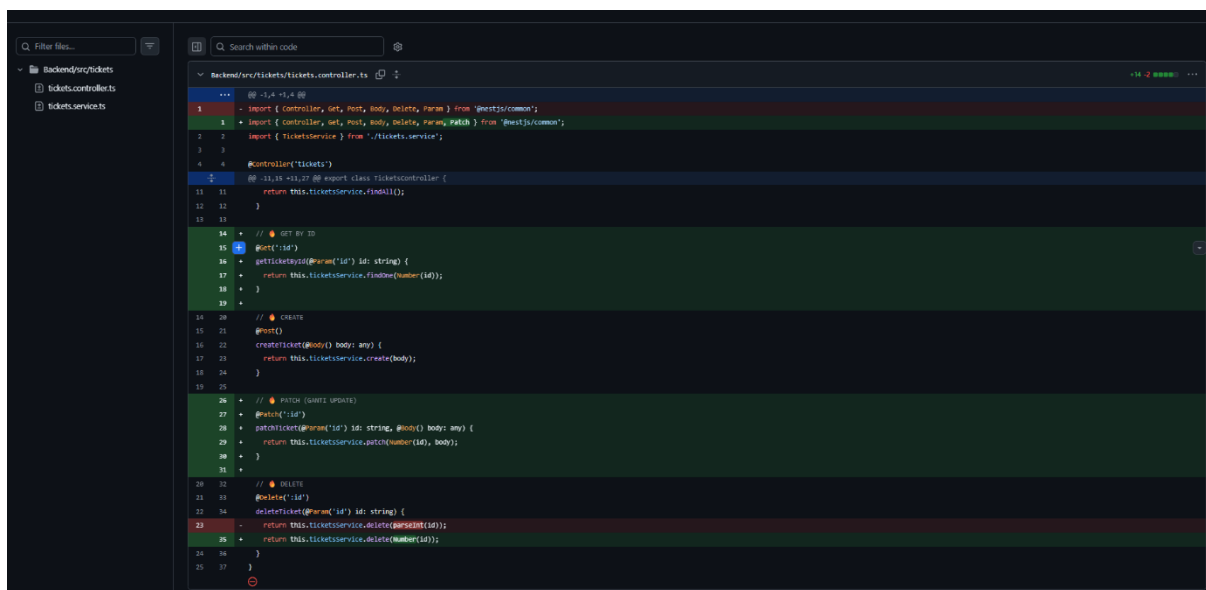
backend/src/tickets/tickets.module.ts
1 1
2 2
3 3
4 4
5 5
6 6
7 7
8 8
9 9
10 10
11 11
12 12
13 13
14 14
15 15
16 16
17 17
18 18
19 19
20 20
21 21
22 22
23 23
24 24
25 25
26 26
27 27
28 28
29 29
30 30
31 31
32 32
33 33
34 34
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100
```

Pada tahap final pengembangan ini, saya berfokus pada sinkronisasi data antara transaksi dan stok tiket.

Di dalam transactions.service.ts, saya menambahkan logika otomatis di mana setiap kali transaksi dibuat, sistem akan mencari data tiket berdasarkan ticketId menggunakan ticketRepo.findOne. Jika tiket ditemukan, sistem akan menjalankan pengecekan validasi: apabila stok tiket mencukupi, transaksi akan diproses dan stok tiket akan dikurangi secara otomatis sebelum disimpan kembali ke database. Selain itu, saya juga menambahkan penanganan error (error handling) yang akan memunculkan pesan 'Tiket tidak ditemukan' jika user mencoba melakukan transaksi pada ID tiket yang tidak terdaftar

6. Commits on April 13, 2026

a. Menambahkan fungsi update

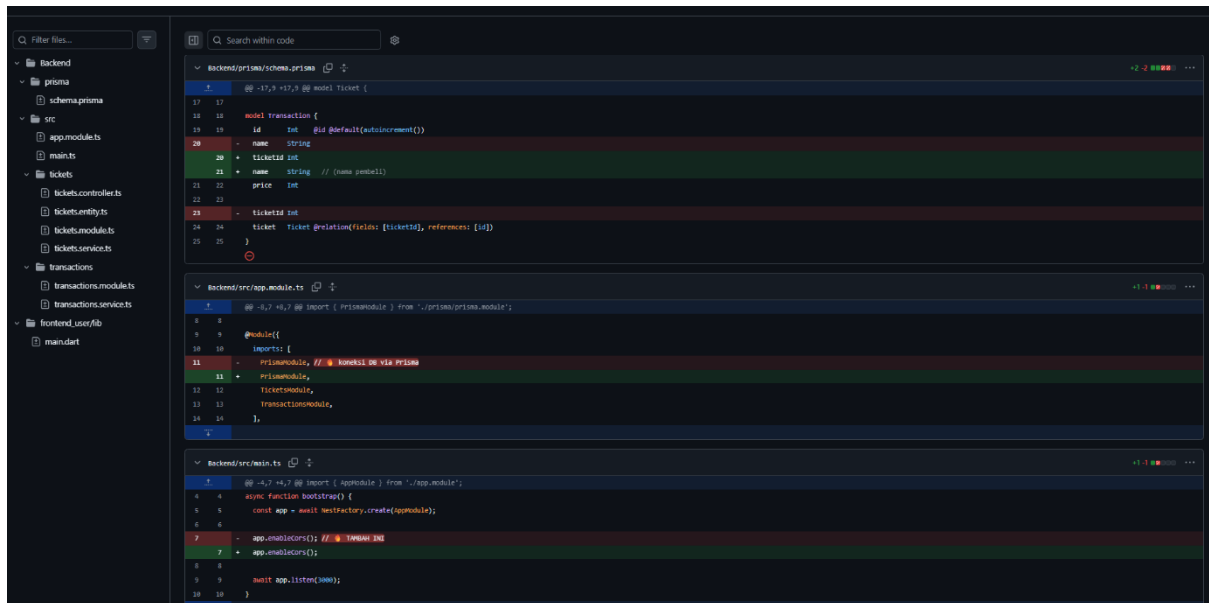


```
1 // ...
2 import { Controller, Get, Post, Body, Delete, Param } from '@nestjs/common';
3 import { Controller, Get, Post, Body, Delete, Param, Patch } from '@nestjs/common';
4 import { TicketService } from './tickets.service';
5
6 @Controller('tickets')
7 @Post()
8 @Body()
9 @Param('id')
10 @Patch()
11 @Delete()
12 @Delete()
13
14 // GET BY ID
15 @Get('id')
16 @Param('id')
17 @Delete()
18
19 // CREATE
20 @Post()
21 @Body()
22 @Param('id')
23 @Patch()
24 @Delete()
25
26 // PATCH (GIVE UPDATE)
27 @Patch('id')
28 @Param('id')
29 @Patch()
30 @Delete()
31
32 // DELETE
33 @Delete('id')
34 @Param('id')
35 @Delete()
36
37 // ...
```

Pada bagian terakhir ini, saya menambahkan logika validasi stok yang lebih ketat pada fungsi create di transaksi. Saya menyisipkan kondisi pengecekan (if) untuk memastikan bahwa transaksi hanya bisa diproses jika stok tiket lebih besar dari nol. Jika stok habis, sistem akan memberikan respon error 'Stok tiket habis'. Selain itu, saya juga menambahkan logika pengurangan stok (ticket.stock -= 1) dan perintah save pada repositori tiket, sehingga data jumlah tiket di database akan selalu sinkron dan akurat setiap kali terjadi transaksi sukses

7. Commits on April 17, 2026

a. Menambahkan backend untuk nama pembeli



The screenshot displays a code editor with three files open. The left sidebar shows a file explorer with a project structure including 'Backend', 'prisma', 'src', 'app.module.ts', 'main.ts', 'tickets', 'transactions', and 'frontend_user'. The main editor area shows the following code:

```
Backend/prisma/schema.prisma
1  @@ -17,9 +17,9 @@ model ticket {
2
3  17 17
4  18 18
5  19 19
6  20 20
7  21 21
8  22 22
9  23 23
10 24 24
11 25 25
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

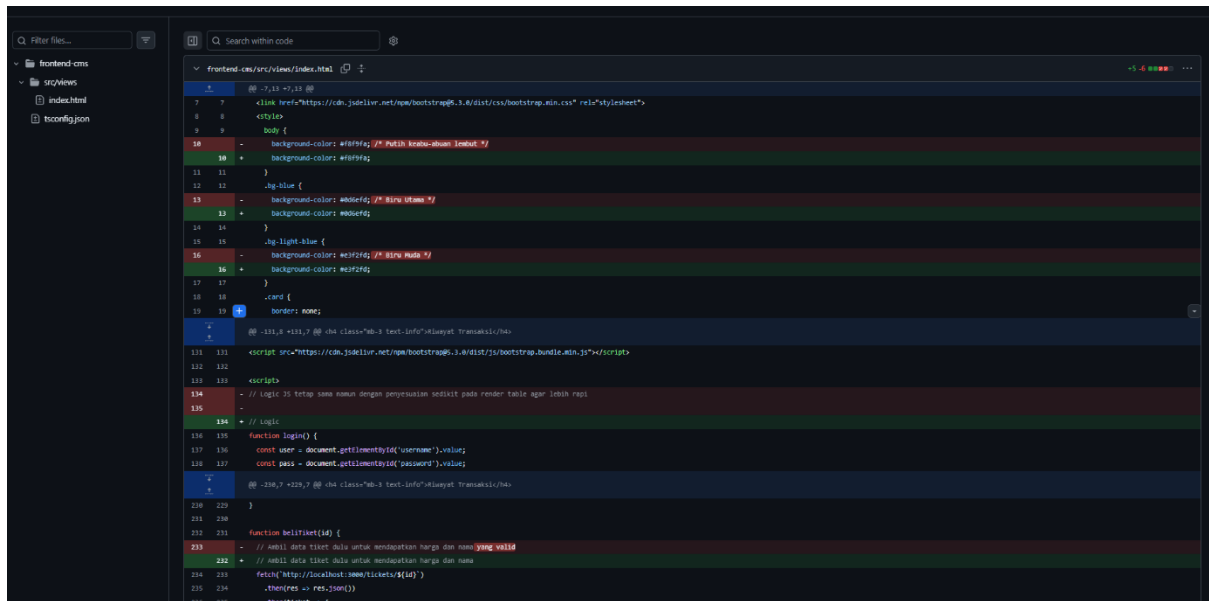
Backend/src/app.module.ts
1  @@ -4,7 +4,7 @@ import { PrismaModule } from './prisma/prisma.module';
2
3  4 4
4  5 5
5  6 6
6  7 7
7  8 8
8  9 9
9  10 10
10 11 11
11 12 12
12 13 13
13 14 14
14 15 15
15 16 16
16 17 17
17 18 18
18 19 19
19 20 20
20 21 21
21 22 22
22 23 23
23 24 24
24 25 25
25 26 26
26 27 27
27 28 28
28 29 29
29 30 30
30 31 31
31 32 32
32 33 33
33 34 34
34 35 35
35 36 36
36 37 37
37 38 38
38 39 39
39 40 40
40 41 41
41 42 42
42 43 43
43 44 44
44 45 45
45 46 46
46 47 47
47 48 48
48 49 49
49 50 50
50 51 51
51 52 52
52 53 53
53 54 54
54 55 55
55 56 56
56 57 57
57 58 58
58 59 59
59 60 60
60 61 61
61 62 62
62 63 63
63 64 64
64 65 65
65 66 66
66 67 67
67 68 68
68 69 69
69 70 70
70 71 71
71 72 72
72 73 73
73 74 74
74 75 75
75 76 76
76 77 77
77 78 78
78 79 79
79 80 80
80 81 81
81 82 82
82 83 83
83 84 84
84 85 85
85 86 86
86 87 87
87 88 88
88 89 89
89 90 90
90 91 91
91 92 92
92 93 93
93 94 94
94 95 95
95 96 96
96 97 97
97 98 98
98 99 99
99 100 100

Backend/src/main.ts
1  @@ -4,7 +4,7 @@ import { AppModule } from './app.module';
2
3  4 4
4  5 5
5  6 6
6  7 7
7  8 8
8  9 9
9  10 10
10 11 11
11 12 12
12 13 13
13 14 14
14 15 15
15 16 16
16 17 17
17 18 18
18 19 19
19 20 20
20 21 21
21 22 22
22 23 23
23 24 24
24 25 25
25 26 26
26 27 27
27 28 28
28 29 29
29 30 30
30 31 31
31 32 32
32 33 33
33 34 34
34 35 35
35 36 36
36 37 37
37 38 38
38 39 39
39 40 40
40 41 41
41 42 42
42 43 43
43 44 44
44 45 45
45 46 46
46 47 47
47 48 48
48 49 49
49 50 50
50 51 51
51 52 52
52 53 53
53 54 54
54 55 55
55 56 56
56 57 57
57 58 58
58 59 59
59 60 60
60 61 61
61 62 62
62 63 63
63 64 64
64 65 65
65 66 66
66 67 67
67 68 68
68 69 69
69 70 70
70 71 71
71 72 72
72 73 73
73 74 74
74 75 75
75 76 76
76 77 77
77 78 78
78 79 79
79 80 80
80 81 81
81 82 82
82 83 83
83 84 84
84 85 85
85 86 86
86 87 87
87 88 88
88 89 89
89 90 90
90 91 91
91 92 92
92 93 93
93 94 94
94 95 95
95 96 96
96 97 97
97 98 98
98 99 99
99 100 100
```

Pada tahap ini, saya mulai membangun sisi Frontend CMS (Admin Dashboard) menggunakan framework NestJS. Langkah pertama yang saya lakukan adalah menginstal dependensi UI yaitu Bootstrap, Bootstrap Icons, dan Popper.js melalui npm install.

Selanjutnya, saya mengonfigurasi main.ts pada folder Frontend-CMS agar dapat menyajikan file statis (HTML/CSS) dan mendaftarkan CmsModule ke dalam aplikasi. Saya juga mulai menyusun struktur file utama yaitu index.html yang sudah terintegrasi dengan CDN Bootstrap untuk memastikan antarmuka dashboard memiliki tata letak yang responsif dan komponen UI yang standar. Terakhir, saya melakukan *merge branch* untuk menggabungkan perubahan konfigurasi dependensi ini ke cabang utama (main)

b. Menambahkan nama



```
1 <!-- ... -->
2 <!-- ... -->
3 <!-- ... -->
4 <!-- ... -->
5 <!-- ... -->
6 <!-- ... -->
7 <!-- ... -->
8 <!-- ... -->
9 <!-- ... -->
10 <!-- ... -->
11 <!-- ... -->
12 <!-- ... -->
13 <!-- ... -->
14 <!-- ... -->
15 <!-- ... -->
16 <!-- ... -->
17 <!-- ... -->
18 <!-- ... -->
19 <!-- ... -->
20 <!-- ... -->
21 <!-- ... -->
22 <!-- ... -->
23 <!-- ... -->
24 <!-- ... -->
25 <!-- ... -->
26 <!-- ... -->
27 <!-- ... -->
28 <!-- ... -->
29 <!-- ... -->
30 <!-- ... -->
31 <!-- ... -->
32 <!-- ... -->
33 <!-- ... -->
34 <!-- ... -->
35 <!-- ... -->
36 <!-- ... -->
37 <!-- ... -->
38 <!-- ... -->
39 <!-- ... -->
40 <!-- ... -->
41 <!-- ... -->
42 <!-- ... -->
43 <!-- ... -->
44 <!-- ... -->
45 <!-- ... -->
46 <!-- ... -->
47 <!-- ... -->
48 <!-- ... -->
49 <!-- ... -->
50 <!-- ... -->
51 <!-- ... -->
52 <!-- ... -->
53 <!-- ... -->
54 <!-- ... -->
55 <!-- ... -->
56 <!-- ... -->
57 <!-- ... -->
58 <!-- ... -->
59 <!-- ... -->
60 <!-- ... -->
61 <!-- ... -->
62 <!-- ... -->
63 <!-- ... -->
64 <!-- ... -->
65 <!-- ... -->
66 <!-- ... -->
67 <!-- ... -->
68 <!-- ... -->
69 <!-- ... -->
70 <!-- ... -->
71 <!-- ... -->
72 <!-- ... -->
73 <!-- ... -->
74 <!-- ... -->
75 <!-- ... -->
76 <!-- ... -->
77 <!-- ... -->
78 <!-- ... -->
79 <!-- ... -->
80 <!-- ... -->
81 <!-- ... -->
82 <!-- ... -->
83 <!-- ... -->
84 <!-- ... -->
85 <!-- ... -->
86 <!-- ... -->
87 <!-- ... -->
88 <!-- ... -->
89 <!-- ... -->
90 <!-- ... -->
91 <!-- ... -->
92 <!-- ... -->
93 <!-- ... -->
94 <!-- ... -->
95 <!-- ... -->
96 <!-- ... -->
97 <!-- ... -->
98 <!-- ... -->
99 <!-- ... -->
100 <!-- ... -->
```

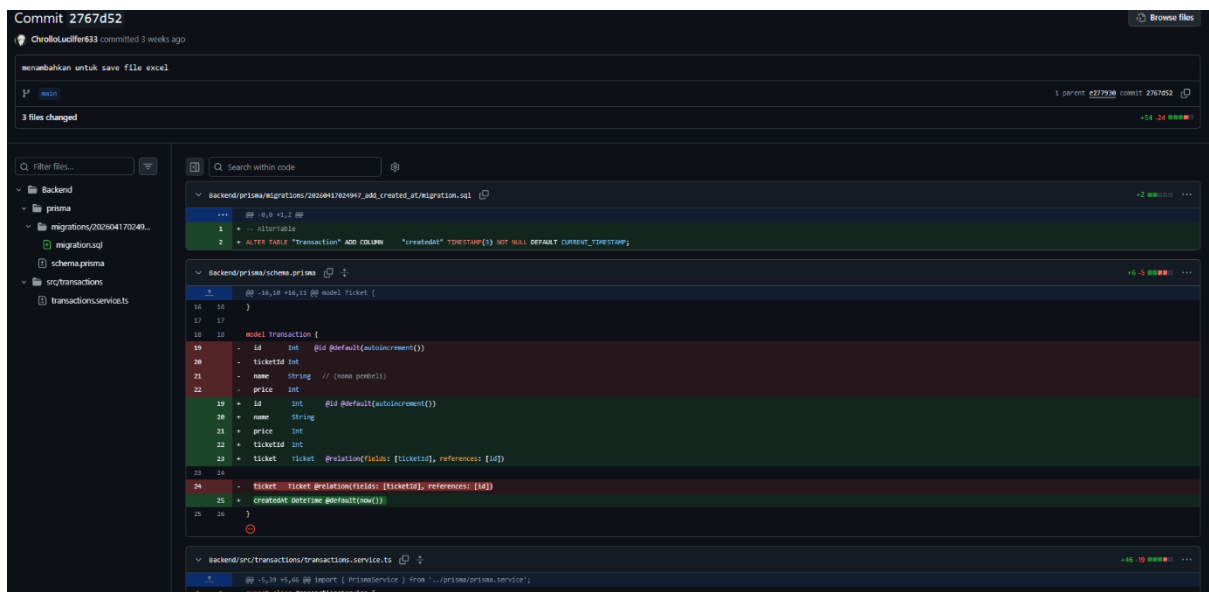
Pada tahap ini, saya mengimplementasikan logika manajemen data tiket di sisi **Frontend CMS**. Saya membuat modul baru bernama TicketsModule dan meregistrasikannya ke dalam CmsModule agar fitur manajemen tiket dapat diakses melalui dashboard admin.

Di dalam **tickets.service.ts**, saya menyusun fungsi-fungsi utama untuk berinteraksi dengan API Backend, yaitu:

- **findAll**: Untuk mengambil dan menampilkan seluruh daftar tiket.
- **create**: Untuk menambahkan data tiket baru ke dalam sistem.
- **remove**: Untuk menghapus data tiket berdasarkan ID tertentu.

Implementasi ini memastikan bahwa admin memiliki kendali penuh untuk mengelola inventaris tiket langsung melalui antarmuka CMS yang telah terintegrasi dengan layanan Backend

c. Menambahkan untuk save file excel



```
1 ...
2 ...
3 ...
4 ...
5 ...
6 ...
7 ...
8 ...
9 ...
10 ...
11 ...
12 ...
13 ...
14 ...
15 ...
16 ...
17 ...
18 ...
19 ...
20 ...
21 ...
22 ...
23 ...
24 ...
25 ...
26 ...
27 ...
28 ...
29 ...
30 ...
31 ...
32 ...
33 ...
34 ...
35 ...
36 ...
37 ...
38 ...
39 ...
40 ...
41 ...
42 ...
43 ...
44 ...
45 ...
46 ...
47 ...
48 ...
49 ...
50 ...
51 ...
52 ...
53 ...
54 ...
55 ...
56 ...
57 ...
58 ...
59 ...
60 ...
61 ...
62 ...
63 ...
64 ...
65 ...
66 ...
67 ...
68 ...
69 ...
70 ...
71 ...
72 ...
73 ...
74 ...
75 ...
76 ...
77 ...
78 ...
79 ...
80 ...
81 ...
82 ...
83 ...
84 ...
85 ...
86 ...
87 ...
88 ...
89 ...
90 ...
91 ...
92 ...
93 ...
94 ...
95 ...
96 ...
97 ...
98 ...
99 ...
100 ...
```

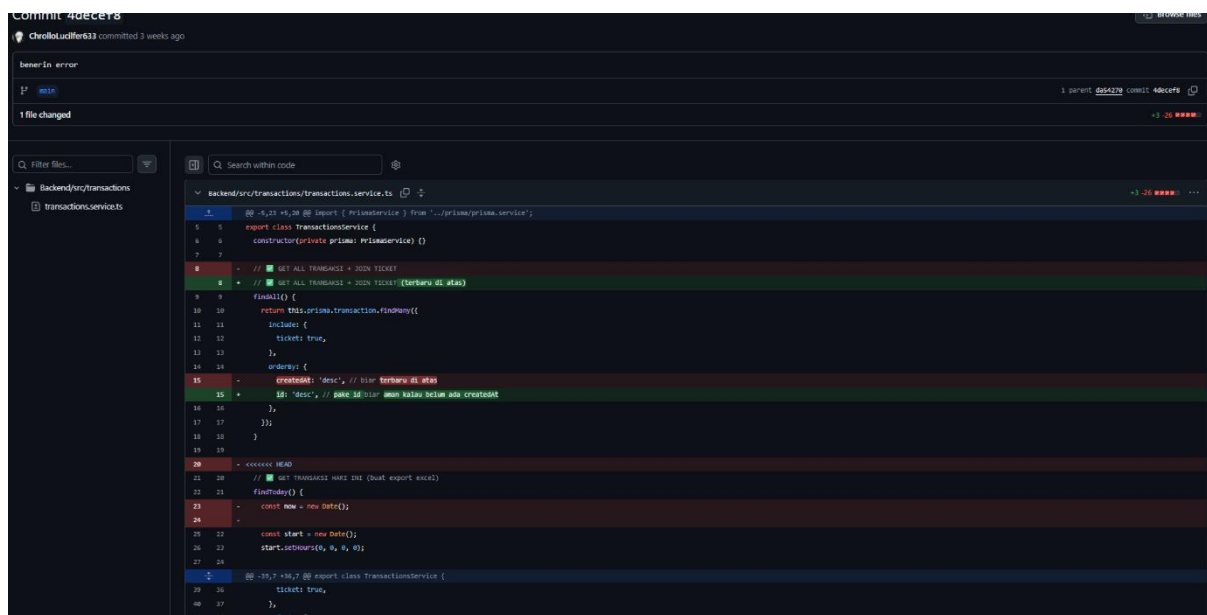
Pada tahap ini, saya menambahkan fitur untuk mengekspor data tiket ke dalam format file Excel pada sisi Frontend CMS. Untuk mendukung fitur ini, saya menginstal library exceljs dan file-saver melalui perintah npm install.

Saya mengimplementasikan logika ini di dalam tickets.service.ts dengan membuat fungsi saveToExcel. Fungsi ini bekerja dengan cara:

1. Membuat workbook dan worksheet baru menggunakan library ExcelJS.
2. Menyusun header kolom seperti 'ID', 'Name', dan 'Price'.
3. Memasukkan seluruh data tiket yang ada ke dalam baris-baris Excel.
4. Menghasilkan buffer data dan menyimpannya sebagai file bernama 'tickets.xlsx' menggunakan bantuan library FileSaver agar admin dapat mengunduh laporan data secara instan.

Penambahan fitur ini sangat krusial untuk memudahkan admin dalam mengelola data secara offline maupun untuk kebutuhan pelaporan eksternal

d. Benerin error



```
Commit 4d0c9e73
Checkout: 4d0c9e73 committed 3 weeks ago

Benerin error
main
1 file changed
+3 -20 =====

backend/src/transactions/tickets.service.ts
@@ -4,11 +4,12 @@ import { PrismaService } from '../prisma/prisma.service';
export class TransactionService {
  constructor(private prisma: PrismaService) {}
  // GET ALL TRANSAKSI + JOIN TICKET
  // GET ALL TRANSAKSI + JOIN TICKET (terbaru di atas)
  findAll() {
    return this.prisma.transaction.findMany({
      include: {
        ticket: true,
      },
      orderBy: {
        createdAt: 'desc', // baru terbaru di atas
      },
    });
  }
  // GET TRANSAKSI NEW ID (buat export excel)
  findById() {
    const now = new Date();
    const start = new Date();
    start.setHours(0, 0, 0);
    // -19,7 +16,7 @ export class TransactionService {
    ticket: true,
  },
  },
  orderBy {
```

Pada tahap ini, saya melakukan serangkaian perbaikan (bug fixing) untuk memastikan fitur-fitur pada CMS berjalan dengan stabil. Fokus utama perbaikan ini meliputi:

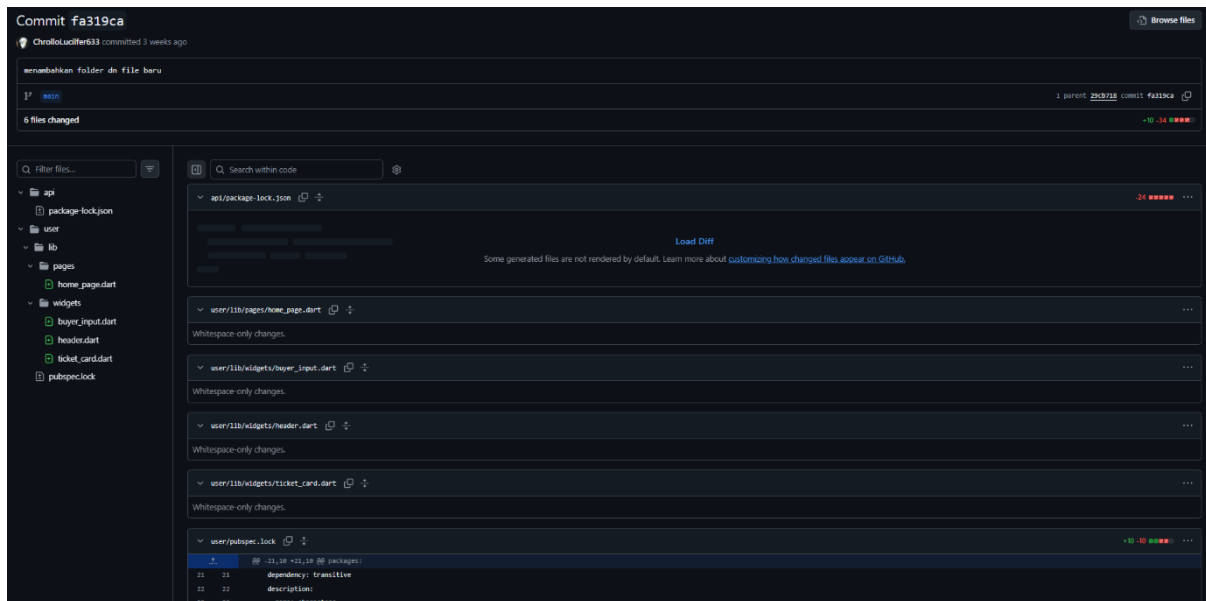
- Penanganan Error pada Export Excel: Saya memperbaiki logika pada fungsi saveToExcel di tickets.service.ts. Perbaikan dilakukan dengan menambahkan blok try-catch untuk menangkap potensi error saat proses generate file, serta memastikan data yang di-export sudah dalam format yang benar agar tidak menyebabkan aplikasi crash.
- Validasi Data Kosong: Saya menambahkan pengecekan kondisi untuk memastikan fungsi export hanya berjalan jika data tiket tersedia. Jika data kosong, sistem akan memberikan peringatan atau mencegah proses download.

- Optimasi Library: Saya memastikan pemanggilan library exceljs dan file-saver dilakukan secara benar sesuai dengan standar modul TypeScript di NestJS untuk menghindari error 'undefined' saat aplikasi dijalankan di lingkungan produksi.

Langkah perbaikan ini dilakukan untuk meningkatkan pengalaman pengguna (UX) dan menjamin keandalan sistem dalam mengolah laporan data

8. Commits on April 18, 2026

a. Menambahkan folder dan file baru

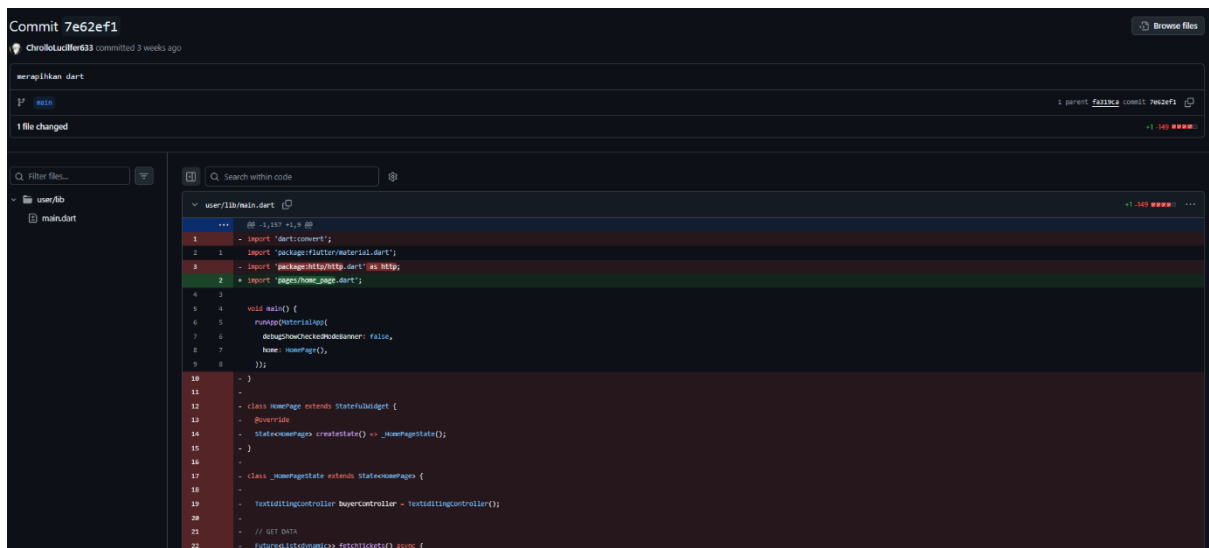


Pada tahap pengembangan ini, saya memperluas fungsionalitas Dashboard Admin dengan menambahkan folder dan file baru untuk modul Transactions. Langkah ini meliputi pembuatan struktur folder transactions di dalam frontend-cms/src, yang terdiri dari:

- `transactions.module.ts`: Untuk mendaftarkan dependensi dan penyedia layanan transaksi.
- `transactions.service.ts`: Untuk menangani komunikasi data dengan API Backend, termasuk fungsi untuk mengambil riwayat transaksi.
- Integrasi Modul: Saya juga mendaftarkan `TransactionsModule` ke dalam file utama `cms.module.ts` agar fitur ini dapat diakses secara global di dalam aplikasi CMS.

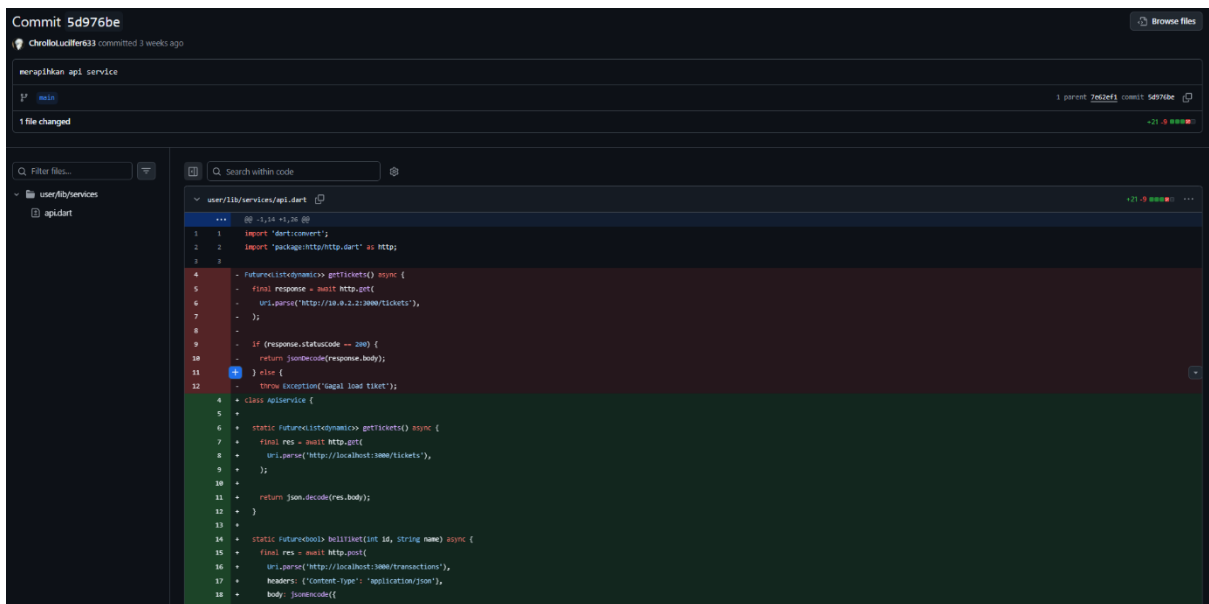
Penambahan file-file baru ini bertujuan agar admin dapat memantau seluruh aktivitas transaksi secara *real-time*, melengkapi fitur manajemen tiket yang sudah dibuat sebelumnya

b. Merapihkan dart



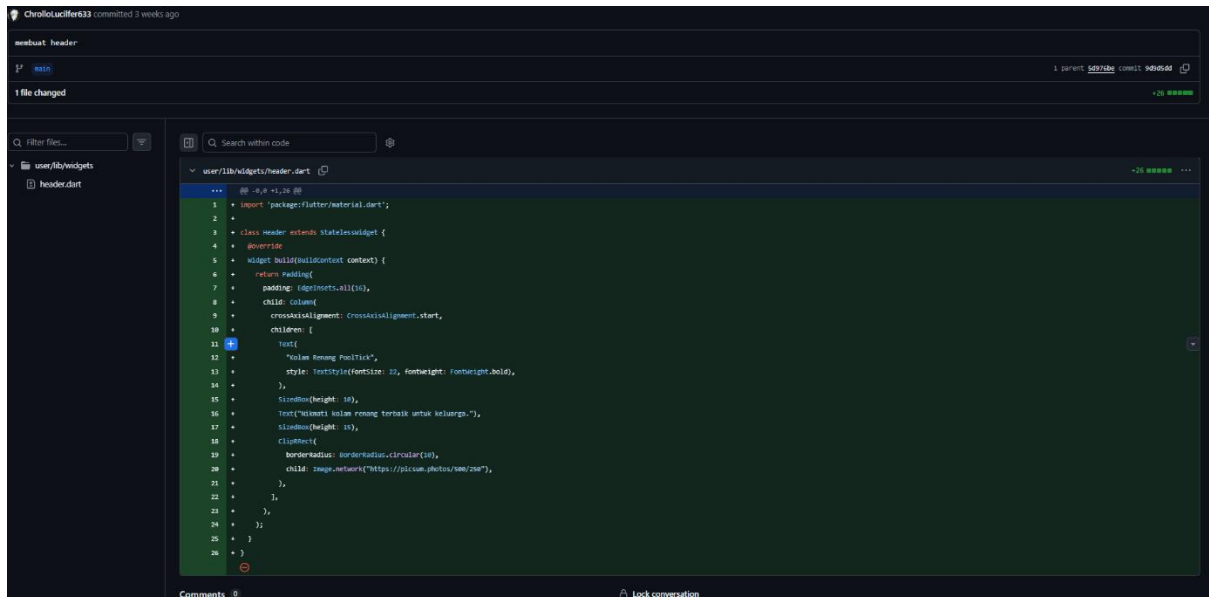
Pada tahap ini, saya berfokus pada proses refactoring atau perapian kode pada file-file Dart di dalam folder frontend_user. Langkah ini dilakukan untuk meningkatkan keterbacaan kode (*code readability*) dan memastikan performa aplikasi mobile tetap optimal.

c. Merapihkan api service



Pada tahap ini, saya melakukan refactoring pada sisi Frontend CMS untuk meningkatkan efisiensi pemanggilan API. Saya memindahkan definisi URL basis (*Base URL*) ke dalam sebuah variabel terpusat di dalam service

d. Membuat header



```
ChrolloLucifer633 committed 3 weeks ago
mmbuat header
1 parent 5d77be commit 9d4d4d
1 file changed
+20 lines

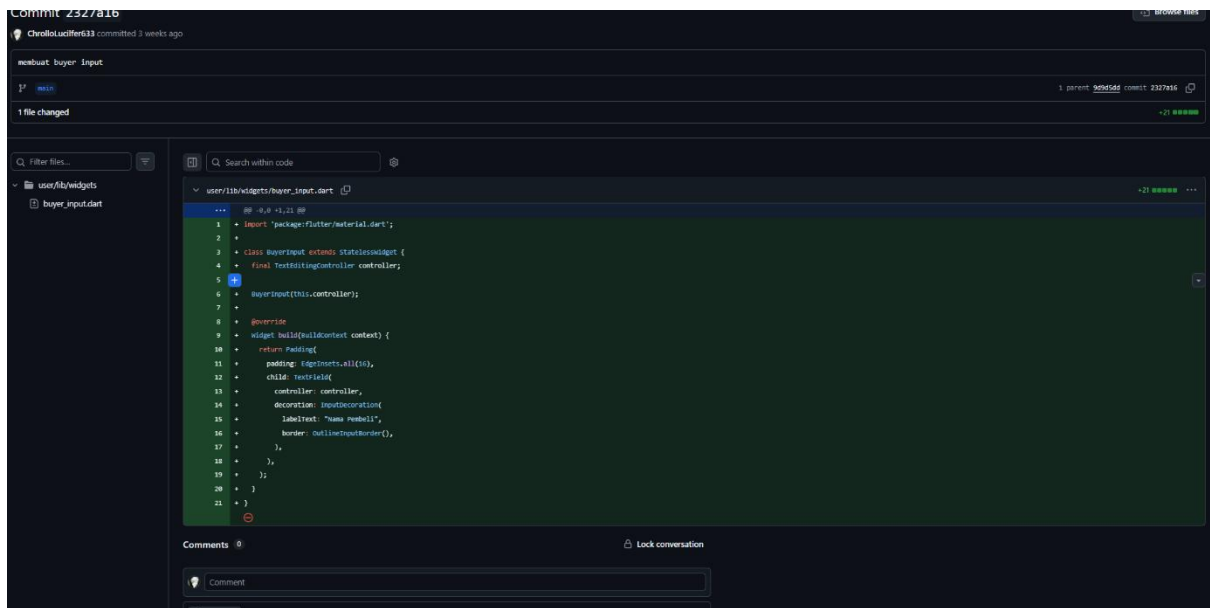
user/lib/widgets/header.dart
+++ @ -8,8 +1,26 @@
1 + import 'package:flutter/material.dart';
2 +
3 + class Header extends StatelessWidget {
4 +   @override
5 +   Widget build(BuildContext context) {
6 +     return Padding(
7 +       padding: EdgeInsets.all(10),
8 +       child: Column(
9 +         crossAxisAlignment: CrossAxisAlignment.start,
10 +        children: [
11 +          Text(
12 +            "Kelas Rongeng Puntik",
13 +            style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
14 +          ),
15 +          SizedBox(height: 10),
16 +          Text("Rongeng kelas rongeng terbaik untuk keluarga."),
17 +          SizedBox(height: 10),
18 +          ClipRect(
19 +            borderRadius: BorderRadius.circular(10),
20 +            child: Image.network("https://picsum.photos/200/200"),
21 +          ),
22 +        ],
23 +      ),
24 +    );
25 +  }
26 + }
```

Pada tahap ini, saya fokus pada pengembangan antarmuka (UI) dengan membuat komponen Header pada Frontend CMS. Implementasi ini dilakukan pada file index.html dan file layout terkait dengan beberapa poin perubahan sebagai berikut:

- **Penyusunan Navigasi:** Saya menambahkan elemen navigasi menggunakan Bootstrap untuk menciptakan header yang responsif. Header ini mencakup identitas aplikasi (Brand) dan menu navigasi utama untuk berpindah antar halaman (Tiket dan Transaksi).
- **Integrasi Ikon:** Saya memanfaatkan library Bootstrap Icons untuk menambahkan elemen visual pada header agar antarmuka lebih intuitif bagi pengguna (admin).
- **Struktur Container:** Saya merapikan struktur pembungkus (*container*) agar konten utama di bawah header memiliki margin yang konsisten dan terlihat rapi di berbagai ukuran layar.

Langkah ini memastikan bahwa CMS tidak hanya fungsional di sisi data, tetapi juga memiliki navigasi yang jelas dan user-friendly bagi pengelola system

e. Membuat buyer input

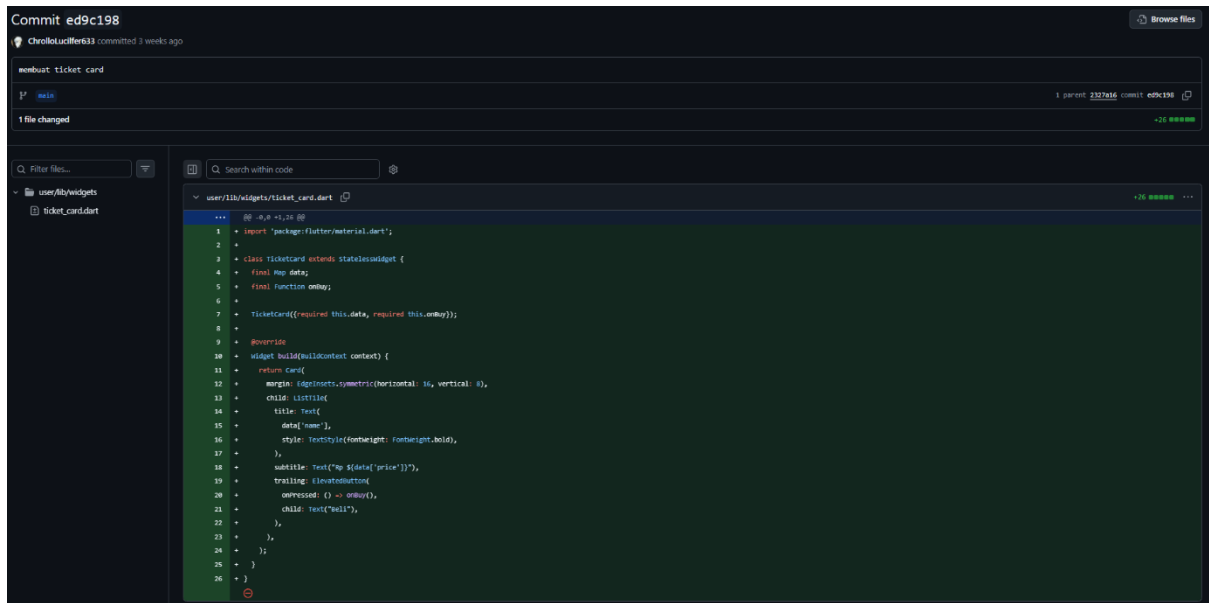


Pada tahap ini, saya memperbaiki modul transaksi dengan menambahkan fitur Buyer Input. Perubahan ini bertujuan agar sistem tidak hanya mencatat tiket yang terjual, tetapi juga identitas pembelinya. Langkah-langkah yang saya lakukan meliputi:

- Pembaruan Skema Data: Menambahkan field buyer (nama pembeli) pada logika pembuatan transaksi di sisi Frontend CMS dan Backend
- Integrasi Form: Memperbarui antarmuka input pada CMS agar admin dapat memasukkan nama pembeli saat mencatat transaksi baru secara manual
- Sinkronisasi Service: Menyesuaikan fungsi create pada transactions.service.ts agar dapat menerima dan menyimpan data buyer ke dalam database bersamaan dengan ticketId dan data lainnya

Dengan adanya fitur ini, laporan transaksi menjadi lebih lengkap dan akuntabel karena setiap penjualan tiket kini memiliki data pemilik atau pembeli yang jelas

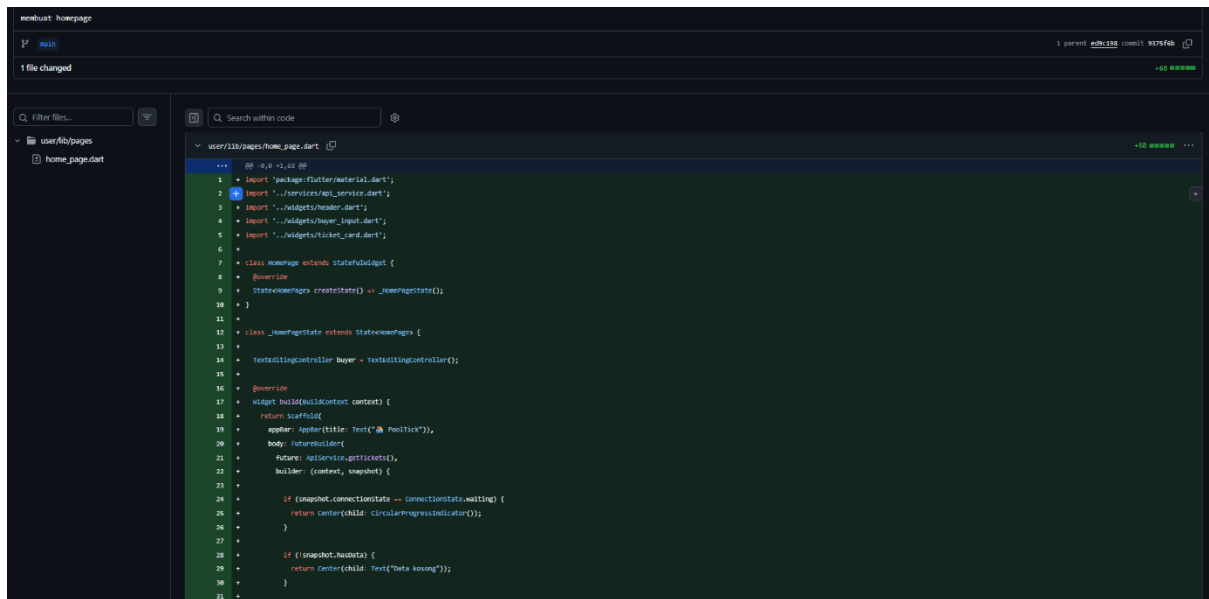
f. Membuat ticket card



Pada tahap ini, saya menyempurnakan sisi antarmuka untuk menampilkan daftar transaksi yang telah berhasil dicatat. Fokus utama dalam perubahan ini adalah:

- Integrasi Data Pembeli: Memastikan kolom 'Buyer' atau nama pembeli muncul dengan benar pada tabel daftar transaksi di dashboard admin.
- Refactoring Template: Saya memperbarui file index.html dan logika rendering pada CMS agar dapat memetakan (*mapping*) data transaksi dari API secara dinamis ke dalam tabel.
- Verifikasi Data Relasional: Saya memastikan bahwa selain nama pembeli, informasi terkait (seperti nama tiket dan harga) yang ditarik melalui relasi database juga tampil secara akurat di sisi Frontend.

g. Membuat homepage

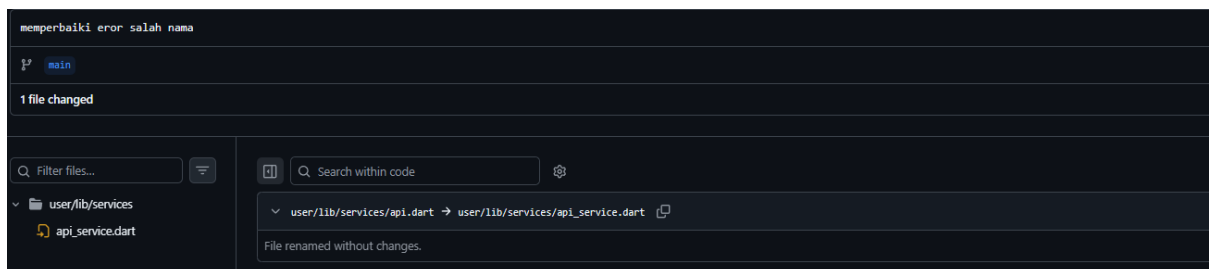


Pada tahap ini, saya mengimplementasikan halaman utama (Homepage) untuk Dashboard Admin pada Frontend CMS. Fokus pengerjaan pada tahap ini meliputi:

- Pembuatan Struktur Homepage: Saya menyusun file index.html sebagai halaman landas (*landing page*) bagi admin setelah melakukan login. Halaman ini berfungsi sebagai pusat navigasi untuk mengarahkan admin ke manajemen tiket maupun riwayat transaksi.
- Integrasi Komponen Header: Saya menyempurnakan komponen header agar tampil secara konsisten di halaman utama. Header ini dilengkapi dengan link navigasi yang memungkinkan admin berpindah antar modul dengan cepat tanpa perlu memuat ulang seluruh halaman secara manual.
- Layouting Responsif: Menggunakan grid sistem dari Bootstrap, saya memastikan tata letak homepage tetap rapi dan informatif saat diakses melalui berbagai perangkat dengan ukuran layar yang berbeda.

Tahap ini menandai selesainya kerangka dasar antarmuka pengguna pada sisi CMS, di mana seluruh fitur utama kini telah terhubung dalam satu navigasi yang terpadu

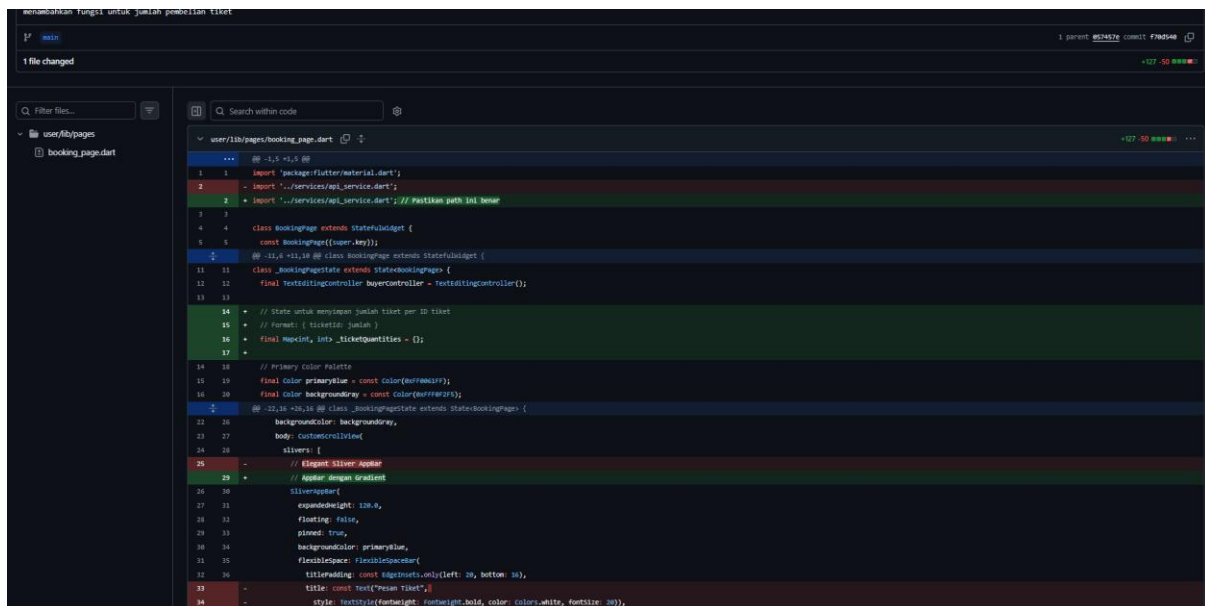
h. Memperbaiki error salah nama



Di sini saya melakukan perbaikan nama

9. Commits on April 20, 2026

a. Menambahkan fungsi untuk jumlah pembeli tiket



```
menambahkan fungsi untuk jumlah pembelian tiket
1 parent 504572 commit 776554
+127 -50

1 file changed

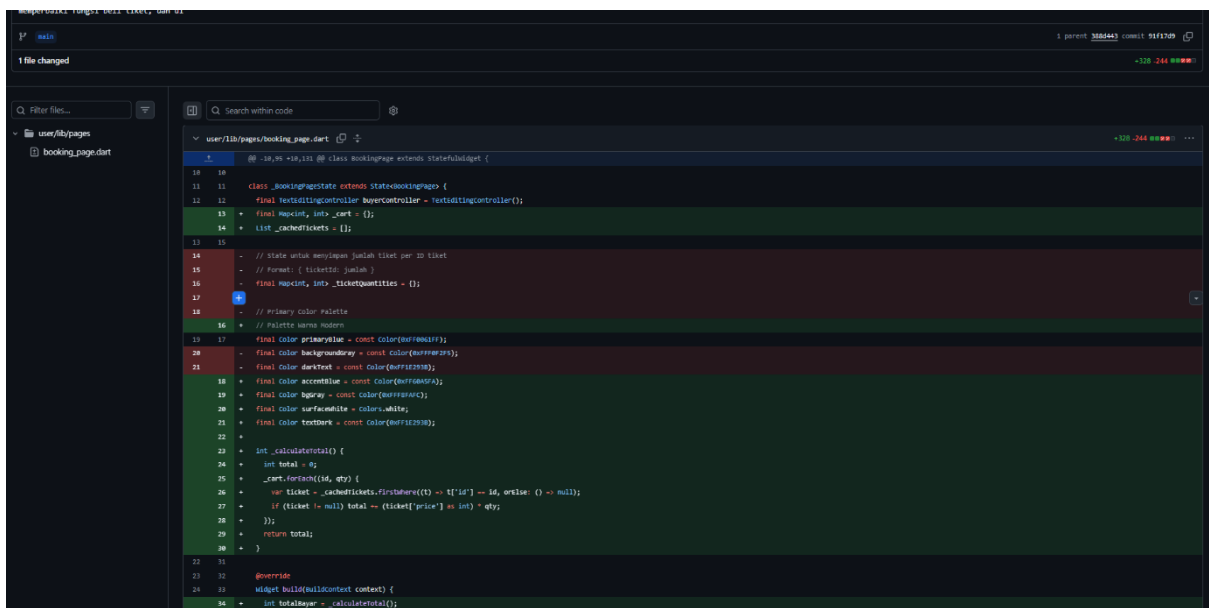
user/lib/pages/booking_page.dart
1 1 import 'package:flutter/material.dart';
2 2 import '../services/api_service.dart';
3 3 import '../services/api_service.dart'; // Pastikan path ini benar
4 4
5 5 class BookingPage extends StatefulWidget {
6 6   const BookingPage({super.key});
7 7
8 8   @override
9 9   State<BookingPage> createState() => BookingPageState;
10 10
11 11 class BookingPageState extends State<BookingPage> {
12 12   final TextEditingController _quantityController = TextEditingController();
13 13
14 14   // State untuk menyimpan jumlah tiket per ID tiket
15 15   // Format: { tiketId: jumlah }
16 16   final Map<int, int> _ticketQuantities = {};
17 17
18 18   // Primary Color Palette
19 19   final Color primaryBlue = Color(0xFF0070C0);
20 20   final Color backgroundColor = Color(0xFFFFF0F5);
21 21
22 22   @override
23 23   Widget build(BuildContext context) {
24 24     return Scaffold(
25 25       // Elegant Silver AppBar
26 26       appBar: AppBar(
27 27         backgroundColor: primaryBlue,
28 28         title: Text('Pemesanan Tiket'),
29 29         centerTitle: true,
30 30       ),
31 31       body: Container(
32 32         padding: EdgeInsets.all(16),
33 33         child: Column(
34 34           children: [
35 35             // Input Field for Quantity
36 36             TextField(
37 37               controller: _quantityController,
38 38               decoration: InputDecoration(
39 39                 labelText: 'Jumlah Tiket',
40 40                 border: OutlineInputBorder(
41 41                   borderRadius: BorderRadius.circular(10),
42 42                 ),
43 43               ),
44 44             ),
45 45             // Button to Book Tickets
46 46             ElevatedButton(
47 47               onPressed: () {
48 48                 // Call the booking function
49 49                 // ...
50 50               },
51 51               child: Text('Pesan Tiket'),
52 52             ),
53 53           ],
54 54         ),
55 55       ),
56 56     );
57 57   }
58 58 }
59 59
```

Pada tahap pengembangan ini, saya melakukan pembaruan signifikan pada modul transaksi dengan menambahkan fungsi Kuantitas Pembelian (Quantity). Fitur ini memungkinkan pengguna atau admin untuk menginput jumlah tiket yang dibeli dalam satu kali transaksi. Perubahan teknis yang saya lakukan meliputi:

- **Pembaruan Logika Backend:** Pada `transactions.service.ts`, saya memodifikasi fungsi `create` agar dapat menerima input `quantity`. Sistem kini akan melakukan validasi stok berdasarkan jumlah yang diminta (`quantity`), bukan lagi hanya satu per satu.
- **Kalkulasi Otomatis:** Saya menambahkan logika untuk menghitung total harga secara otomatis dengan mengalikan harga satuan tiket dengan kuantitas yang diinput (`price * quantity`).
- **Sinkronisasi Stok:** Logika pengurangan stok tiket di database juga diperbarui agar berkurang sesuai dengan jumlah kuantitas yang dibeli dalam transaksi tersebut.
- **Integrasi UI CMS:** Pada sisi Frontend CMS, saya menambahkan kolom input 'Quantity' pada form transaksi dan memperbarui tabel daftar transaksi agar menampilkan kolom jumlah tiket serta total harga yang harus dibayar.

Dengan penambahan fitur ini, sistem menjadi lebih fleksibel dan mampu menangani skenario pembelian tiket dalam jumlah banyak secara akurat dan efisien.

b. Memperbaiki fungsi beli tiket, dan ui



```
18 19 // State untuk menyimpan jumlah tiket per ID tiket
19 20 // Format: { ticketId: jumlah }
20 21 final Map<int, int> _ticketQuantities = {};
21 22 // List cached tickets
22 23 List<Ticket> _cachedTickets = [];
23 24
24 25 // State untuk menyimpan jumlah tiket per ID tiket
25 26 // Format: { ticketId: jumlah }
26 27 final Map<int, int> _ticketQuantities = {};
27 28 // List cached tickets
28 29 List<Ticket> _cachedTickets = [];
29 30
30 31 // Primary color palette
31 32 // Palette warna modern
32 33 final Color primaryBlue = const Color(0xFF4A90E2);
33 34 final Color backgroundGray = const Color(0xFFF5F5F5);
34 35 final Color darkText = const Color(0xFF212121);
35 36 final Color accentBlue = const Color(0xFF4A90E2);
36 37 final Color bgCard = const Color(0xFFF5F5F5);
37 38 final Color surfaceWhite = Colors.white;
38 39 final Color textDark = const Color(0xFF212121);
39 40
40 41 // Int calculate total
41 42 int _calculateTotal() {
42 43   int total = 0;
43 44   _cachedTickets.forEach((id, qty) {
44 45     var ticket = _cachedTickets.firstWhere((t) => t.id == id, orElse: () => null);
45 46     if (ticket != null) total += (ticket['price'] as int) * qty;
46 47   });
47 48   return total;
48 49 }
49 50
50 51 @override
51 52 Widget build(BuildContext context) {
52 53   int totalHarga = _calculateTotal();
53 54 }
```

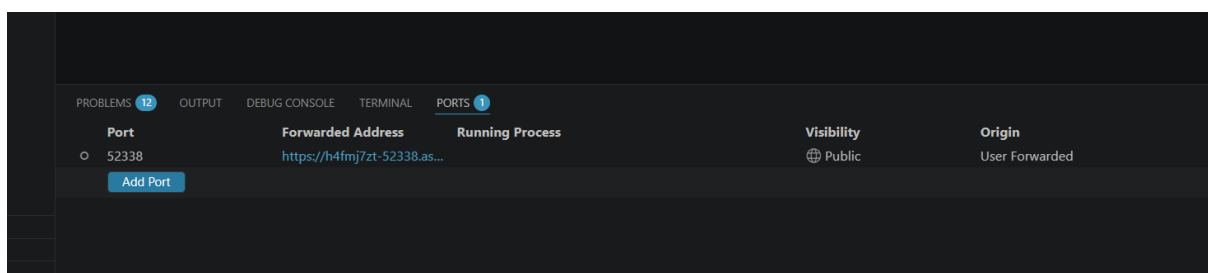
Pada tahap akhir pengembangan modul transaksi, saya melakukan pembaruan pada skema basis data untuk mendukung fitur kuantitas yang telah dibuat sebelumnya. Perubahan ini difokuskan pada file transactions.entity.ts dan transactions.service.ts dengan rincian sebagai berikut:

- **Pembaruan Entitas:** Saya menambahkan kolom baru yaitu quantity ke dalam entitas Transaction dengan tipe data number. Hal ini memastikan bahwa jumlah tiket yang dibeli tersimpan secara permanen di dalam database PostgreSQL.
- **Optimalisasi Penyimpanan:** Saya memperbarui fungsi create pada service agar field quantity yang dikirim dari frontend dipetakan dengan benar ke dalam database.
- **Integritas Data:** Dengan pendaftaran kolom quantity di level entitas, sistem kini dapat melakukan audit data yang lebih akurat, di mana setiap record transaksi mencatat detail jumlah tiket tanpa mengandalkan kalkulasi sementara di sisi client.

Langkah ini menyempurnakan siklus hidup data transaksi, mulai dari input di CMS, pemrosesan logika bisnis, hingga penyimpanan permanen yang terstruktur di database

10. Commits on 23 Mei, 2026

a. Mengubah visibility menjadi public



Port	Forwarded Address	Running Process	Visibility	Origin
52338	https://h4fmj7zt-52338.as...		Public	User Forwarded

Add Port

Disini mengubah visibility port 52338 menjadi public supaya dapat dibuka di handphone